

Received 7 May 2025, accepted 9 June 2025, date of publication 27 June 2025, date of current version 7 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3583584

TOPICAL REVIEW

A Comprehensive Survey on Resource Management for IoT Applications in Edge-Fog-Cloud Environments

NOUR EL HOUDA BOUBAKER¹, KARIM ZAROUR¹, NAWAL GUERMOUCHE²,
AND DJAMEL BENMERZOUQ¹

¹LIRE Laboratory, University of Constantine 2–Abdelhamid Mehri, Constantine 25016, Algeria

²LAAS-CNRS, INSA, University of Toulouse, 31042 Toulouse, France

Corresponding author: Nour El Houda Boubaker (nour.boubaker@univ-constantine2.dz)

This work was supported by the French National Research Agency (ANR) LabEx International Center for Mathematics and Computer Science in Toulouse (CIMI) through French State Programme “Investissements d’Avenir” under Grant ANR-11-LABX-0040.

ABSTRACT Resource management in Edge-Fog-Cloud environments has emerged as a fundamental challenge in modern distributed computing systems, primarily due to the heterogeneous nature of underlying infrastructure, highly dynamic workloads, and the need to optimize multiple, often conflicting, Quality of Service (QoS) metrics such as latency, energy consumption, cost, and security. These environments are characterized by their layered architecture and real-time decision-making demands, which require efficient coordination among distributed resources to meet the expectations of both users and applications. Although several surveys in the literature have explored aspects of resource management, they tend to focus narrowly on specific components such as resource allocation, task scheduling, or workload placement, often neglecting the interdependence of these components. Moreover, a considerable number of surveys fail to consider the rapidly evolving landscape of edge and fog computing, where dynamic environmental changes and fluctuating user demands are becoming the norm. To bridge these gaps, this survey provides a comprehensive and up-to-date review of resource management strategies in Edge-Fog-Cloud ecosystems. We adopt a holistic perspective that integrates both combinatorial optimization techniques and learning-based methods, highlighting their roles in addressing different facets of resource management. The survey delves into five key aspects: task scheduling, resource placement, allocation, offloading, and service migration. We propose a structured taxonomy categorizing over 70 works (2017-2025) by objectives, techniques, QoS decision-making mechanisms, and environmental factors such as user mobility and network variation. Special attention is given to the synergy between combinatorial and Machine Learning-based approaches, identifying where each excels and under which conditions their integration can yield more robust solutions. Ultimately, this survey outlines the current limitations of the field, uncovers open challenges, and suggests future research directions to enable more intelligent, efficient, and adaptive resource management solutions.

INDEX TERMS Fog computing, edge computing, resource allocation, scheduling, offloading, migration, machine learning, combinatorial optimization.

I. INTRODUCTION

The Internet of Things (IoT) paradigm has rapidly gained acceptance in the development of smart environments across various domains. With the rise of IoT-enabled Cyber-Physical

Systems (CPSs) in smart cities, healthcare, and Industry 4.0, real-time processing services are necessary to handle the massive amounts of produced data. The Cloud-centric execution of IoT applications fails to meet such requirements as Cloud data centers reside at a multi-hop distance from the IoT devices [1], [2]. This leads to an increase in communication latency between IoT devices and Cloud data

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta¹.

centers, which can have a significant adverse impact on the performance and in general on the overall Quality of Service (QoS) of IoT applications. Fog and Edge computing have emerged as innovative technologies to complement Cloud computing. By leveraging the computing capabilities within the local network, these technologies enable computation tasks to be performed closer to the IoT devices rather than Cloud [3]. This leads to significant improvements in the overall performance of IoT applications. While Fog and Edge Computing offer tremendous opportunities for IoT applications, such as reduced latency, improved data security, and efficient network utilization, they also give rise to several challenges. In fact, Fog and Edge systems exhibit characteristics such as mobility, location awareness, low latency, real-time interaction support, and geo-distribution across urban areas [4]. However, they are also constrained by limited computational and power capacities, making resource management a significant challenge in Fog and Edge-based systems.

In the context of resource management within Fog-Edge-Cloud environments, ensuring QoS is of paramount importance. QoS refers to the performance and reliability characteristics of a system as experienced by users or applications [5]. Consequently, discovering and implementing an optimal configuration of resources that effectively meets the QoS requirements of IoT applications stands as a pivotal challenge. This challenge becomes even more pronounced when considering the highly dynamic and time-varying nature of parameters associated with the environment in which these IoT applications operate. Factors such as fluctuating network conditions, varying workload demands, and the dynamic mobility patterns of users contribute to the complexity of achieving an optimal resource configuration that can consistently uphold QoS standards [6], [7], [8]. Thus, addressing these challenges and investigating the different strategies for resource allocation in response to dynamic environmental conditions becomes imperative for ensuring the seamless performance and reliability of IoT systems across Fog, Edge, and Cloud computing environments.

Over the years, considerable efforts have been devoted to grappling with the intricate challenges of resource management across a spectrum of dimensions including resource allocation, placement, scheduling, offloading, and migration. This concerted concern has been driven by the overarching goal of attaining optimal or near-optimal solutions to the myriad complexities inherent in resource management within Edge-Fog-Cloud Computing environments. The paramount objective throughout these endeavors has been to uphold a superior QoS not only for the applications themselves but also for the end users who interact with these applications.

Central to these endeavors have been the extensive utilization and exploration of Combinatorial Optimization-based techniques, which encompass a diverse array of methodologies such as linear programming, heuristics, and meta-heuristics, each contributing uniquely to the complex landscape of resource management. These techniques have

played a pivotal role in crafting efficient resource allocation strategies, optimizing resource placement, and fine-tuning scheduling mechanisms to enhance overall system performance and QoS [9], [10]. These techniques, characterized by their ability to explore vast solution spaces and derive near-optimal solutions [11], [12], have played a pivotal role in shaping resource management strategies across diverse applications and use cases.

Moreover, the landscape of resource management has witnessed a significant surge in interest and exploration of Machine Learning (ML) algorithms, marking a transformative shift in how computational resources are managed within Edge-Fog-Cloud Computing environments. This burgeoning interest has been fueled by the remarkable capabilities and potential that ML paradigms offer in optimizing resource utilization and enhancing system performance. The utilization of ML paradigms within this context is characterized by its extensive and diverse spectrum. This encompasses a wide array of methodologies, each serving distinct purposes: Supervised Learning (SL), where algorithms acquire knowledge from labeled data to facilitate predictions or decision-making; Unsupervised Learning (USL), where algorithms uncover underlying patterns and structures within unlabeled datasets [13]; Deep Learning (DL), which entails intricate neural networks capable of extracting intricate representations from extensive datasets [14]; Reinforcement Learning (RL), a method focusing on decision sequences to attain long-term objectives through iterative learning processes; and Deep Reinforcement Learning (DRL) [15], a fusion of deep learning techniques and reinforcement learning principles, tailored for intricate decision-making scenarios [16], [17], [18].

A. RELATED SURVEYS AND MOTIVATION

Over the past few years, several research studies have emerged in this field. For instance, [19], [20], [21], [22], [23] focus on the integration of ML techniques primarily within Edge and Fog computing paradigms, exploring how Artificial Intelligence (AI) can enhance resource management and decision-making processes in these distributed environments. For instance, [19] highlights key AI techniques applied to enhance IoT intelligence, discussing progress across various layers, including perception, network, and application. While their focus is broad spanning domains like smart homes, healthcare, and industry. The work provides valuable insights into how AI enables intelligent decision-making and autonomy in IoT environments. However, it does not delve specifically into resource management challenges such as scheduling, placement, or migration in Edge-Fog-Cloud environments. Reference [20] provides a broad overview of Fog computing concepts and explores the integration of ML techniques in this context. While it offers valuable insights into general applications and challenges, its primary focus is on foundational concepts rather than resource management. Reference [22] introduces a comprehensive

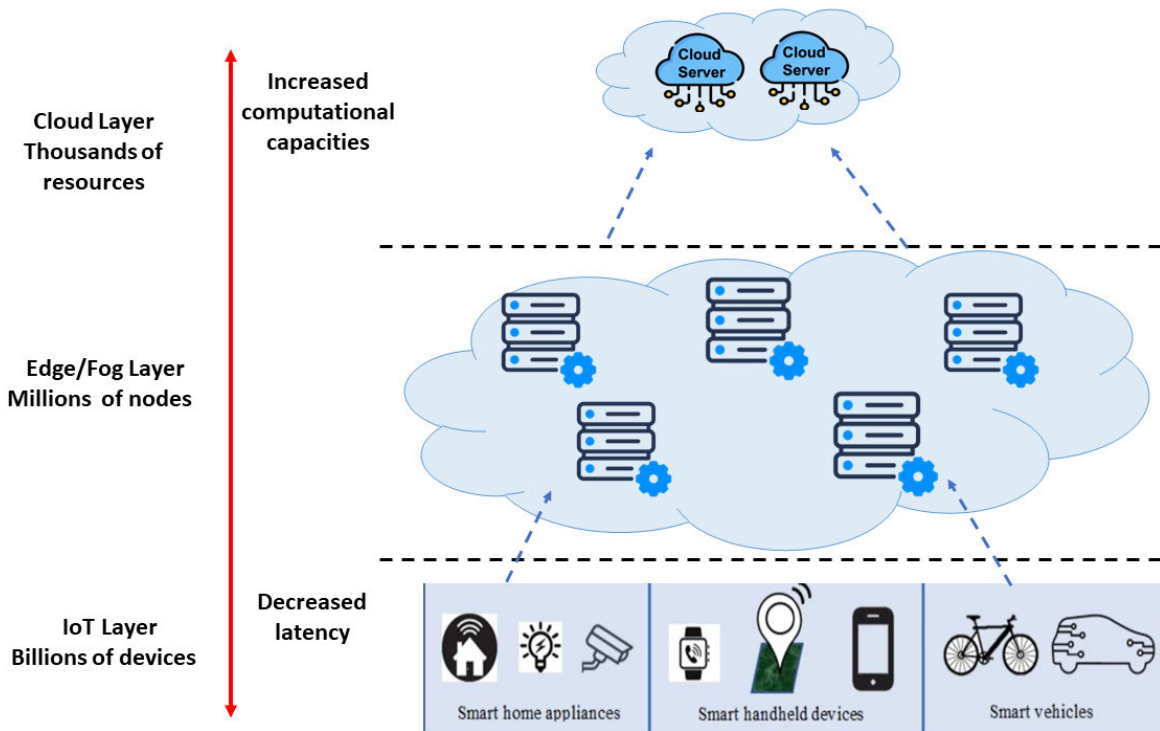


FIGURE 1. Edge/Fog/Cloud layers.

review of how edge computing enhances AIoT (Artificial Intelligence of Things) by enabling intelligent services closer to end devices. It categorizes recent research into key areas such as AI model deployment at the edge, system architectures, and application scenarios, emphasizing the role of edge computing in supporting low-latency and privacy-sensitive AI tasks. While the survey highlights the synergy between AI and edge computing, it does not extensively cover resource management, making it complementary to our focus on resource orchestration in dynamic Edge-Fog-Cloud environments. Reference [21] provides a broad overview of the role of AI in shaping the future of computing paradigms, including Cloud, Fog, Edge, and Quantum computing. While it offers valuable insights into AI-driven techniques and their applicability across various emerging infrastructures, its focus is more conceptual and futuristic rather than centered on the granular aspects of resource management. Reference [24] presents a comprehensive review of resource management approaches in Fog computing, with a particular focus on combinatorial optimization techniques. The study categorizes existing solutions based on key resource management aspects such as placement and scheduling. While offering valuable insights into optimization-based methods, the work does not explore AI-driven strategies or the integration of dynamic environmental features, which limits its applicability to more adaptive and intelligent systems. The work [25] introduces a survey of optimization algorithms for service placement in Fog computing environments. The study primarily

focuses on analyzing classical and metaheuristic optimization techniques applied to placement problems, emphasizing their role in improving system performance through efficient resource utilization. While it offers a solid foundation on placement strategies, it does not cover other essential aspects such as scheduling, offloading, or dynamic system behaviors, nor does it explore the potential of AI-driven or learning-based approaches. Reference [26] provides a comprehensive analysis of service placement strategies in Cloud environments, emphasizing key optimization objectives such as latency, energy consumption, and cost. The study classifies various placement techniques, primarily focusing on combinatorial and heuristic-based approaches, while also identifying critical challenges like scalability and heterogeneity. However, the work is limited to Cloud contexts and does not extend its analysis to Edge or Fog layers, nor does it explore the integration of learning-based methods or dynamic environment features. The survey conducted in [27] presents a systematic mapping study that reviews existing research on resource allocation in Fog computing. The authors structured their investigation around four key research questions: identifying the primary objectives of resource allocation, categorizing the algorithms used classified into Exact, Heuristic-based, and Meta-heuristic-based approaches, determining whether the proposed solutions adopt centralized or decentralized architectures, and highlighting the most frequently considered QoS metrics for achieving optimal resource allocation. Similarly, [28] provided an overview of scheduling methods and

algorithms used in Fog computing, classifying approaches into Heuristic, Meta-Heuristic, and Hybrid groups. The study discusses approaches that aim to optimize resource utilization, minimize latency, and improve overall system performance. In a previous study [29], we conducted a question-based survey to examine service migration within Edge/Fog computing environments using machine learning techniques. The classification was based on key criteria, including migration type, migration policy, migration techniques, and the QoS metrics involved. Reference [30] surveys computation offloading in Uncrewed Aerial Vehicles (UAVs)-enabled Mobile Edge Computing (MEC). The authors explore various strategies and challenges related to offloading tasks in UAV-assisted MEC environments, focusing on optimizing resource management, reducing latency, and enhancing energy efficiency. They examine different offloading techniques, including binary, partial, and load relaying approaches. The authors provided a comparative analysis of these methods and highlighted their strengths and weaknesses. The authors in [31] provide a comprehensive survey on mobility-aware Fog computing in dynamic networks with mobile nodes. Their work emphasizes how mobility impacts service continuity, latency, and resource availability, reviewing key approaches for handling dynamic changes in network topology. While their focus lies in mobility management and its implications on Fog computing, our work complements this perspective by broadening the scope to encompass multiple aspects of resource management, while also revealing other dynamic features that influence system performance. Additionally, [32] conducts a systematic overview of existing task scheduling techniques for Cloud-Fog environments, categorizing papers into ML-based, heuristic-based, metaheuristic-based, and deterministic mechanism categories. The survey categorizes existing approaches based on scheduling objectives, methods, and architectural considerations, with a particular emphasis on latency, energy consumption, and resource utilization. However, it focuses primarily on scheduling in isolation, without extensively addressing other key aspects such as offloading, migration. Reference [33] presents a comprehensive survey on AI-driven resource management strategies in Fog and Edge computing, particularly tailored for IoT applications. Their work classifies resource management techniques across various AI paradigms, focusing on scheduling, allocation, and offloading. While the survey offers valuable insights into AI-based methods, it lacks a detailed exploration of combinatorial optimization techniques and does not examine the interplay between AI and combinatorial optimization approaches in dynamic environments. Reference [23] surveys on how AI augments Edge and Fog computing, focusing on improving scheduling and system maintenance. They identify emerging trends such as federated learning, self-adaptive systems, and lightweight AI models. The paper also discusses major challenges including data heterogeneity, scalability, privacy preservation, and energy efficiency, and suggests future

directions to build more intelligent and resilient Edge-Fog ecosystems. A recent study by [34] provides a comprehensive overview of ML-based computation offloading in MEC environments, focusing on the integration of ML models to optimize offloading decisions in resource-constrained and latency-sensitive mobile applications. The study analyzes how ML contributes to improving QoS metrics under various deployment environments. It categorizes existing research based on architectural setups (e.g., number of MEC servers/base stations), decision timing, offloading objectives, and learning techniques, and highlights their trade-offs.

Despite the existing surveys provide a valuable analysis of the literature, they present the following limits:

- **Limited Holistic View:** Most surveys tend to address specific issues such as placement, scheduling, resource allocation, offloading, or migration in isolation, overlooking the interdependencies among these components and the nuanced differences in their objectives.
 - **Emphasis on Optimization-Based Approaches:** Most of resource management studies focus on optimization-based methods, overlooking predictive approaches and limiting the exploration of diverse strategies that could be effective in various contexts.
 - **Overlooking Dynamic Features:** Existing surveys often ignore dynamic factors like fluctuating workloads and changing user demands, resulting in covering strategies that may not adapt well to real-time changes.
 - **Lack of Integration between Combinatorial and ML Strategies:** Current literature often treats combinatorial optimization and ML as disjoint domains. None of them explore the potential synergies between them or provide guidance on when to employ each technique depending on system conditions or application requirements.
 - **Absence of Recent Works:** The lack of recent studies and dynamic resource management initiatives limits the relevance and scope of the insights provided.
- Given these limitations, this survey aims to fill the gap by offering a more comprehensive analysis of resource management in Edge-Fog-Cloud environments. It will focus on the interplay between Combinatorial Optimization and ML techniques, exploring where each is most beneficial and how they can be integrated to address both static and dynamic challenges in resource management. Furthermore, it will examine the dynamic features of modern systems such as fluctuating workloads, user behavior, and real-time changes, and how these impact resource allocation and scheduling. By providing a more holistic and integrated approach, this survey will contribute to the raise attention of more adaptive and efficient resource management solutions in modern computing environments.

B. CONTRIBUTIONS AND PAPER ORGANIZATION

In this study, we present a focused and in-depth survey on the application of Combinatorial Optimization and ML techniques for resource management in Edge-Fog-Cloud environments. Our objective is to provide a comprehensive

TABLE 1. Comparison of surveys on resource management in Edge-Fog-Cloud environments.

Works	Year	Emerging Computing Paradigm	Focus on Resource Management problems	Resource Management Aspects Covered (Placement /Resource Allocation/ Scheduling Offloading Migration)	AI-driven Search	CO-driven Search	QoS Optimization	Exploration of dynamic environments features	CO vs ML Focus Analysis
[20]	2019	Fog	×	×	✓	×	×	×	×
[24]	2020	Fog	✓	P/S/O/M	×	✓	✓	×	×
[25]	2020	Fog	✓	P/S/M	×	✓	✓	×	×
[27]	2020	Fog	✓	P/RA/S/O	×	✓	✓	×	×
[19]	2020	Cloud/Fog/Edge	×	×	✓	×	×	×	×
[26]	2021	Cloud	✓	P/S/M	×	✓	✓	×	×
[28]	2021	Fog	✓	S	×	✓	✓	×	×
[22]	2021	Edge	×	×	✓	×	×	×	×
[29]	2022	Fog/Cloud	✓	O/M	✓	×	✓	×	×
[30]	2022	MEC	✓	O	✓	×	✓	×	×
[21]	2022	Cloud/Fog/Edge/Quantum	×	×	✓	×	×	×	×
[32]	2023	Fog	✓	S	✓	✓	✓	×	×
[33]	2023	Fog/Edge	✓	P/RA/S/O	✓	✓	✓	×	×
[31]	2023	Fog	✓	O/M	✓*	✓*	✓*	×	×
[23]	2023	Edge/Fog	✓	S	✓	✓*	✓*	×	×
[34]	2024	MEC	✓	O	✓*	✓	×	×	×
This Survey	2024	Cloud/Fog/Edge	✓	P/RA/S/O/M	✓	✓	✓	✓	✓

Legend: ✓ = Yes; × = No; ✓* = Partially

and up-to-date overview of recent advancements addressing key challenges such as task scheduling, service placement, resource allocation, computation offloading, and task migration.

What distinguishes this work is its integrated perspective to the best of our knowledge, this is the first survey that systematically investigates where Combinatorial Optimization and ML techniques are most frequently applied, highlighting their application across various resource management. Unlike previous studies that often examine these techniques in isolation, we analyze their intersections and trade-offs. Moreover, we place particular emphasis on which of these solutions are designed to adapt to dynamic and heterogeneous environments, which remains a critical yet underexplored dimension in existing literature.

- Identifying the different underlying features of resource management approaches.
- Developing a comprehensive taxonomy that classifies these approaches based on a range of criteria, including the techniques employed, optimized QoS metrics, and the nature of the considered environment.
- Summarizing the latest approaches in the field of resource management.
- Presenting a statistical discussion that makes the differences between Combinatorial Optimization and ML approaches.
- Discussing the related challenges of the existing approaches.

The structure of the paper is outlined in Figure 2. The research and paper selection methodology is presented in Section II. Section III highlights the dimensions of resource management, including the aspects and the different computing environments. Section IV discusses resource management techniques. Classification based on the optimized QoS metrics and the type of environment are presented in Section V. Section VI presents a comparative discussion of reviewed approaches and their limitations as well as the future research challenges to address in the field of resource management. Finally, Section VII concludes the paper.

II. RESEARCH METHODOLOGY AND TAXONOMY

In this section, we start by presenting our research methodology, which is an important step for constructing a well-structured and insightful knowledge paper. By following [35], our research methodology is mainly composed of research questions and a paper selection method. Then, we introduce our proposed taxonomy.

A. RESEARCH QUESTIONS

This study aims to collect and analyze all relevant papers related to resource management approaches in the Edge, Fog, and Cloud aimed at optimizing QoS. Given the significant objective of the current survey, which is to explore Combinatorial Optimization and ML techniques, it is essential to address several key research questions to deal with related concerns. The research questions that we will try to answer through this survey are described as follows:

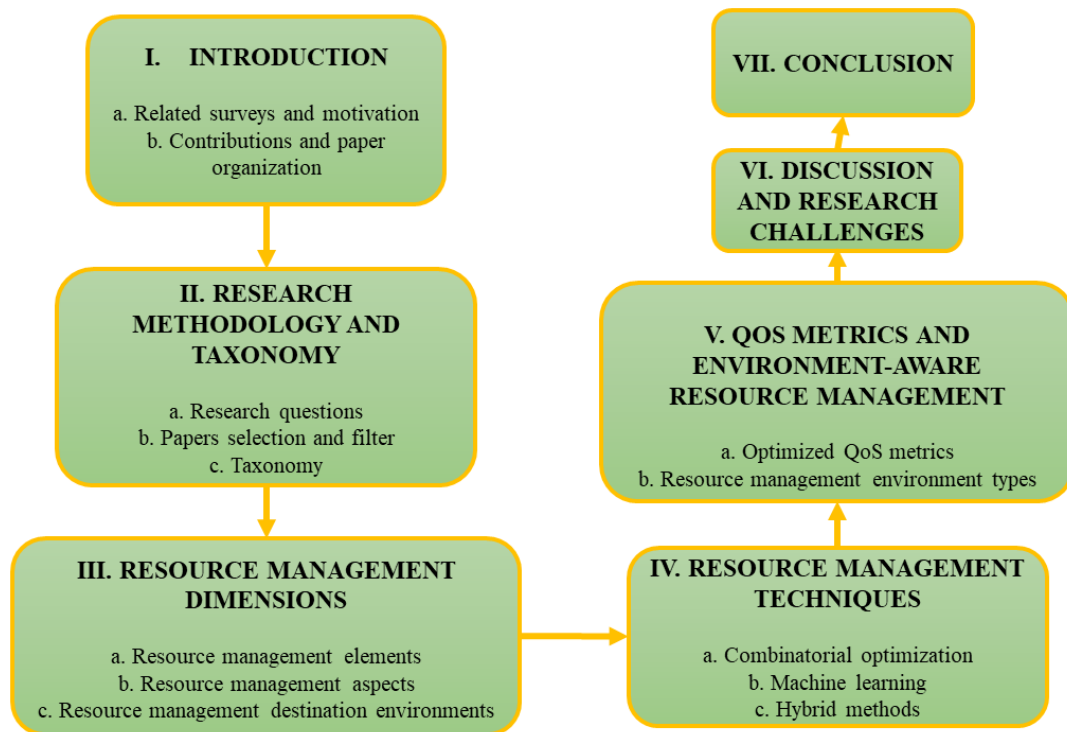


FIGURE 2. Overview of the survey structure.

- **Q1:** What are the different techniques of Combinatorial Optimization and ML used to optimize resource management? What is the technique the most used in the two fields?
- **Q2:** Are there specific resource management aspects where Combinatorial Optimization techniques are more commonly employed compared to ML?
- **Q3:** Are Combinatorial Optimization techniques more commonly utilized than ML techniques for managing specific element in resource management?
- **Q4:** What are the diverse QoS metrics prioritized and optimized in the studied papers? Are there particular QoS metrics where ML techniques are more frequently utilized for optimization compared to Combinatorial Optimization?
- **Q5:** Do these approaches mostly consider a static or a dynamic environment? What are the different features considered in dynamic environments? Which approach, Combinatorial techniques, or ML, offers greater adaptability when dealing with dynamic changes in environment? Considering the changing environmental conditions over time, does this imply that resource allocation remains static or becomes dynamic?
- **Q6:** What are the application domains of these approaches?
- **Q7:** What is the deployment environment the most considered in the literature?
- **Q8:** What computational tools are typically used to implement Combinatorial approaches compared to ML for resource management optimization?

- **Q9:** Are experimental settings based on synthetic data or real data? How does the scalability of resources and tasks differ between Combinatorial Optimization and ML-based approaches in resource management? Is the consideration of static and dynamic environments a critical aspect in scalability?
- **Q10:** What are the major limitations and challenges that exist in this field? Are there unique difficulties when integrating Combinatorial Optimization and ML techniques for optimizing resource management?

B. PAPERS SELECTION AND FILTER

This subsection elaborates on the review's search strategy: the scope (i.e., publication period and publication venue), search method, search string, inclusion / exclusion criteria, and quality assessment criteria.

1) SEARCH SCOPE

We conducted our selection across databases like IEEE Xplore, Springer, and Google Scholar, focusing on papers from 2017 to 2024. This period marked a significant increase in research on Edge/Fog Computing, ML, and related topics.

2) SEARCH STRING

Based on the research questions in Section II-A, we identified key search terms to select relevant papers. These terms fall into three groups: the study topics, the deployment environments, and the types of techniques used. Our selection phrase is thus presented as follows:

(“*Placement” OR “*Resource Allocation” OR “*Migration” OR “*Scheduling” OR “*Offloading” OR “*Deployment”) AND (“Fog” OR “Edge” OR “Cloud”) AND (“Combinatorial Optimization” OR “Heuristic” OR “Meta-Heuristic” OR “Exact” OR “Machine Learning” OR “Supervised” OR “Unsupervised” OR “Deep” OR “Reinforcement” OR “Deep Reinforcement”).

We used the asterisk character (*) for the first term group in order to capture possible term variations such as service, task, application, and workflow.

3) INCLUSION/EXCLUSION CRITERIA

Then, the filter phase is applied, in which we evaluate and investigate the selected papers in the previous step by applying exclusion (EC)/inclusion (IC) criteria to title, abstract, and full paper. Firstly, we exclude papers that meet any of the following EC:

- **EC1:** The abstract and title do not contain any of the selection terms.
- **EC2:** The papers addressing Combinatorial Optimization or ML techniques combined with other techniques (e.g. Multi-Criteria Decision-Making methods)
- **EC3:** The papers that consider the Quality of Experience (QoE)
- **EC4:** Workshop articles, position papers, keynotes, reviews, tutorials, and panel discussions.
- **EC5:** Short papers (less than 6 pages).
- **EC6:** The extended papers that have similar contributions and objectives.
- **EC7:** The study does not answer at least nine questions.

Our main goal is to review papers on Combinatorial Optimization and ML techniques related to resource management. We use **EC2** to filter out papers focusing mainly on techniques other than these, like Multi-Criteria Decision-Making methods. This helps us stay focused on our core interests. Also, our survey critically evaluates QoS-based strategies while using **EC3** to exclude papers solely on QoE, keeping our review aligned with our fundamental interests.

Next, we define our IC as follow:

- **IC1:** The papers that consider QoS optimization.
 - **IC2:** The papers that mostly address CPS and IoT.
 - **IC3:** The papers are written in English.
- CPS and IoT have a significant impact on resource management strategies as they are integral components of modern computing paradigms, especially in Edge and Fog environments. By applying **IC2**, we provide practical insights and solutions that can be applied directly to real-world scenarios.

4) QUALITY ASSESSMENT CRITERIA

Finally, we define Quality Assessment Criteria (QAC) to ensure that the selected papers get along with our main objectives.

- **QAC1:** Papers must have a clearly articulated problem statement and well-defined objectives, ensuring that the research direction aligns with the focus of our study.

- **QAC2:** Papers should provide a comprehensive description of the proposed solution or methodology, along with clear findings and results.
 - **QAC3:** The paper should adhere to standard academic writing conventions, including clear organization, proper citation of sources, and adherence to ethical guidelines.
- As a result of two previous steps (selection and filter), we obtained 71 papers. Figure 3 captures the repartition per year of the studied works since 2017.

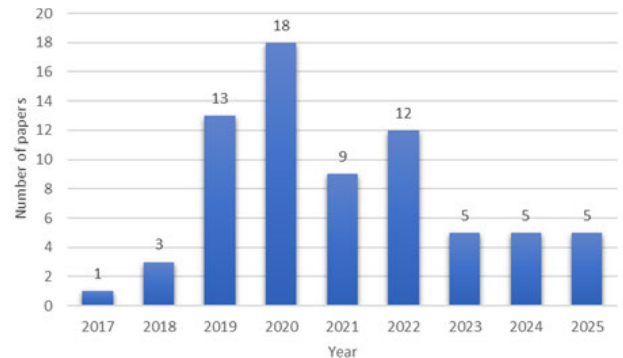


FIGURE 3. The number of reviewed papers per year.

C. TAXONOMY

Figure 4 provides a taxonomy of resource management highlighting the main considered components:

- **Dimensions of resource management:** It includes the elements involved in resource management, the aspects of resource management, and the target computing environment. The elements can be a monothetic application, independent tasks, dependent tasks, or data. Resource management aspects include placement/allocation, scheduling, offloading, or migration. The target environment can be a single or hybrid.
- **Used techniques:** We mainly consider three dominant categories, which are Combinatorial Optimization, ML-based, and Hybrid methods.
- **Optimized metrics:** They are classified into four main categories which are performance, cost, energy consumption, and security.
- **Environment type:** Which refers either to static or dynamic environments.

In the following, we detail each component.

III. RESOURCE MANAGEMENT DIMENSIONS

In this section, we discuss the different types of elements considered in resource management problems. We also explain the main differences between resource management aspects and destination environments. Thereafter, a classification of approaches based on these features is provided.

A. RESOURCE MANAGEMENT ELEMENTS

It determines the nature of the element concerned by the resource management approach. We could distinguish four categories, which are:

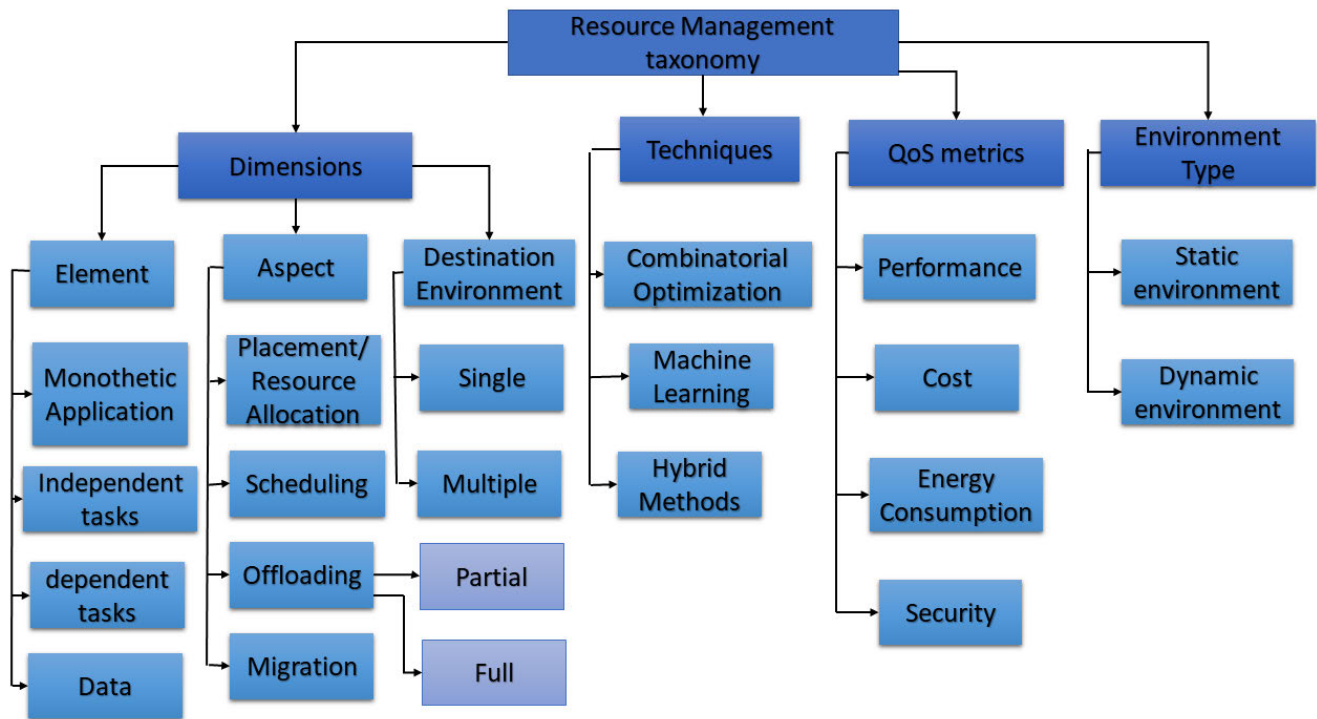


FIGURE 4. Taxonomy of resource management.

- **Monolithic Application:** It denotes an approach where an entire application is treated as a single unit, without being divided into smaller components. Such applications are commonly addressed in migration strategies tied to user mobility situations. For example, as a user transitions from one geographical area to another, the monolithic application must be shifted from one server to another to maintain uninterrupted access and functionality [36], [37].
- **Independent tasks:** Resource management strategies often address applications characterized by independent tasks, where each task operates autonomously without communication or dependencies on other tasks. The ready time of these tasks is typically determined by their arrival time, allowing them to be processed independently [6], [38], [39]. This model is particularly applicable in domains where a high volume of tasks generated from user devices [39], [40], [41], [42].
- **Dependant tasks :** Unlike the previous category, this one considers a set of tasks with logical dependencies between them, where a task can be started only after its dependent tasks are all finished. Generally, a Directed Acyclic Graph (DAG) is used to model a dependant task-based application [43], [44], [45]. Each DAG vertex represents a task and the edges between vertices refer to the interdependencies between tasks. In the literature, workflow-based resource management is less addressed compared to independent tasks-based resource management. It is difficult to achieve an optimal solution that satisfies the computational and QoS requirements of each task and

considers the dependencies of tasks [46], [47], [48]. Dependent tasks are common in industrial IoT (IIoT) environments such as smart manufacturing and assembly lines, where current task execution depends on previous tasks completion [49], [50].

- **Data:** In daily life, sensors that are spread across different geographical locations gather a significant volume of data, such as information pertaining to air quality. The processing of this data involves the need for effective filtering, aggregation, processing, and storage. As part of this process, the collected data is transmitted to Fog nodes, which carry out filtering, aggregation, and basic analysis tasks. In some cases, the data may also be forwarded from Fog nodes to the Cloud, where more resource-intensive analysis, computationally expensive tasks, and extensive storage operations are conducted [51]. Data-centric resource management plays a central role in smart city and healthcare applications such as traffic flow optimization and environmental monitoring where continuous and scalable data handling is essential [52], [53].

B. RESOURCE MANAGEMENT ASPECTS

Resource management covers many aspects as presented in Figure 5. Although their major objective is to find the optimal resources for hosting various applications, each aspect is used in a specific context. In literature, four aspects are widely investigated, and they are described as follows:

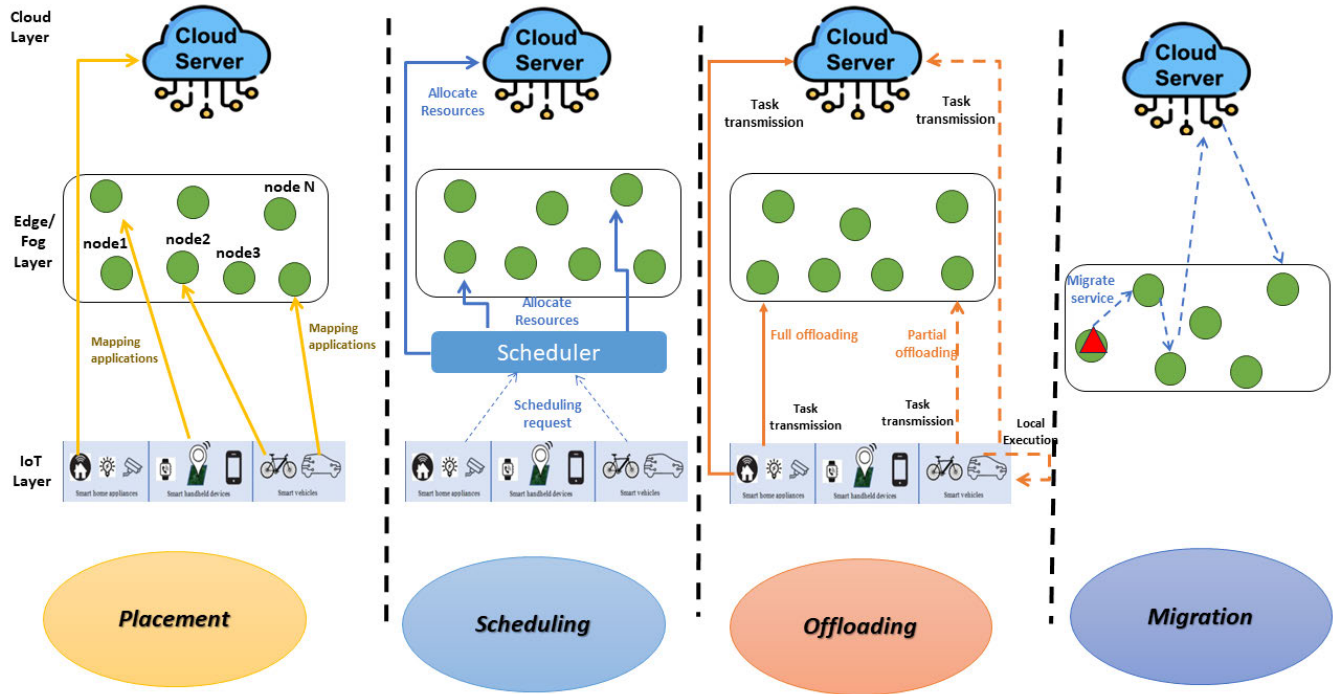


FIGURE 5. Resource management aspects.

- Placement /Resource Allocation:** It is a mechanism that provides the facility of mapping different user tasks to available and suitable resources in order to meet the required specifications in terms of QoS factors such as cost, time constraints, and energy. It is worth noting that in the literature, the term “Placement” mainly refers to resource allocation. Within our taxonomy, these approaches fall under the same category as resource allocation.
- Scheduling:** Scheduling refers to the process of allocating computing resources to an application and mapping constituent components of that application onto those resources in order to meet certain QoS and resource conservation goals [54]. The task scheduler is responsible for selecting the appropriate resources over a defined period of time for performing the received tasks to give a suitable result to the user. The concept of scheduling has a great similarity to resource allocation. Both aspects aim to allocate the appropriate resources for performing the tasks. However, in scheduling, the scheduler also has to determine in which order each task should be executed on the resource [55].
- Offloading:** It is a mechanism that consists of transferring tasks, services, applications, or data from resources with limited capacities to others, which can provide enough computing capacity, storage space, and a relatively fast response time to achieve an optimal experience in the quality of applications. We could distinguish two categories of offloading: **full offloading** and **partial offloading**. The first consists of sending the entire element to the selected nodes for computation, while, in partial offloading, a part of the element executes locally [56].
- Migration:** It is a solution that enables services and applications to be moved from initial resources to other resources. According to [57], a migration problem can be defined as the issue of determining when, where, and how to move a service from one node to another node in order to achieve the desired compromise among all QoS factors under consideration. Even if migration and offloading are closer, they are different and allow to handle different issues. Offloading occurs to meet the computational and energy requirements of hardware-limited and battery-powered devices. Whereas, migration is mainly conducted with the aim of preventing QoS degradation due to many reasons such as continuous user mobility, Fog mobility, or dynamic changes in request patterns [58]. Continuous user mobility is usually considered when a user moves from a zone covered by a base station to another, and at each move step, the element has to be migrated to follow the end device. On the other hand, Fog mobility is caused by the continuous movement of Fog nodes. Finally, in dynamic request patterns, the element has to migrate continuously according to the dominant request locations [29]. Table 2 provides a clear comparison that highlights the principal differences among these concepts.

C. RESOURCE MANAGEMENT DESTINATION ENVIRONMENTS

Resource management approaches may be divided into two categories based on the destination environment in which the element has to be placed, which can be either single or hybrid as captured in Figure 6. In a single destination, only one layer is targeted, which may be Edge Computing, Mobile Edge

TABLE 2. Differentiation between Scheduling, Offloading, Placement, and Migration.

Concept	Definition	Key Objective	Real-Time Example	Key Differences
Placement / Resource Allocation	Mapping user tasks to suitable resources to meet QoS (cost, time, energy).	Efficiently assign tasks to the best available resources.	A cloud provider assigns a new VM to a physical server with enough CPU/RAM for a customer's web application [59].	• Static, one-time decision at deployment. • Does not consider execution ordering or dynamic load. • Focuses on initial mapping, not subsequent adjustments.
Scheduling	Allocating resources to tasks in a specific order to meet QoS and efficiency goals.	Determine when and in what order tasks should execute on resources.	In a CPS scenario, fog computing enables local processing of tasks from wearable sensors. A dynamic scheduler assigns data-intensive tasks to nearby fog nodes to reduce transfer time, while compute-intensive tasks go to high-capacity nodes for faster execution. This minimizes latency, eases network load, and supports real-time services [60].	• Involves ordering and timing of task execution, not just assignment. • Can adapt to real-time workload changes.
Offloading	Transferring tasks/data from resource-limited devices to more powerful nodes (edge/Fog/Cloud).	Reduce latency, save energy, or enhance performance.	A smartphone offloads a application tasks such as face recognition task to a nearby edge server instead of processing it locally [42], [61].	• Initiates execution remotely without prior local run. • Balances trade-off between communication cost and compute benefit.
Migration	Moving a running service/application from one node to another dynamically.	Maintain QoS by adapting to changes in mobility, load distribution, or service-level requirements.	The cyber tasks of an industrial robot are migrated from one Fog server to another as the user moves into a different geographical region [29], [49].	• Involves relocating already running services (unlike offloading, which starts execution elsewhere) • Triggered by resource overload or user mobility.

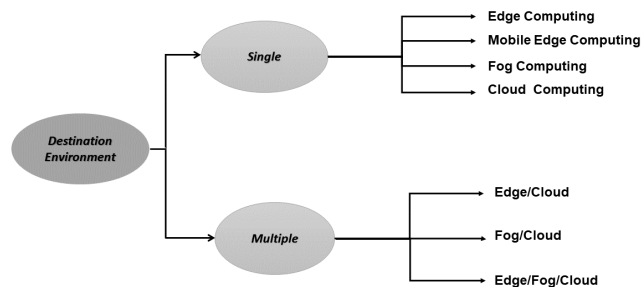


FIGURE 6. Resource management destination environment.

Computing (MEC), Fog Computing, or Cloud Computing. They are defined as follows:

- **Edge Computing:** It represents a novel computing paradigm characterized by its proximity to end-users. As described in [62], it encompasses the set of technologies that empower computation to take place at the network's

edge. This involves processing downstream data in support of Cloud services and handling upstream data for IoT services.

- **Mobile Edge Computing:** It can be defined as an implementation of Edge Computing to bring computational and storage capacities to the edge of the network within the Radio Access Network to reduce latency and improve context awareness. The MEC nodes or servers are usually co-located with the Radio Network Controller or a macro base station [63]. The MEC servers are responsible for providing network real-time status information such as network capacity and loaded tasks, along with information about user devices such as location and data of transactions.
- **Fog Computing:** The OpenFog Consortium defines Fog Computing as a system-level horizontal architecture that distributes resources and services of computing, storage, control, and networking anywhere along the continuum from the Cloud to things [64]. Edge Computing is more or

less the same as Fog Computing in terms of low latency, low bandwidth costs, mobility support, high scalability, and virtualization service. However, it has more limited resources, limited computation and storage capabilities, and proximity to end devices than Fog [65]. Moreover, Fog Computing relies on interconnection capabilities among nodes, whereas Edge Computing runs in isolated Edge nodes [66].

- **Cloud Computing:** According to the National Institute of Standards and Technology (NIST) [67], Cloud Computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. Even though Cloud Computing has numerous advantages in terms of cost savings, efficiency, scalability, and reliability, it also has some challenges when it deals with a massive amount of data, such as high latency, load balancing, energy consumption, security, and privacy.

Unlike the single environment type, the hybrid one relies on multiple layers. Mostly, Fog/Edge nodes are prioritized to guarantee a high QoS for the considered services/tasks. However, when the computational capacities provided by Fog/Edge nodes do not fulfill the awaited requirements, Cloud Computing is then considered.

To summarize, a comparison of resource management approaches based on the type of element, the handled aspect, and the destination environment is given in Table 3

IV. RESOURCE MANAGEMENT TECHNIQUES

Figure 7 provides an overview of the techniques utilized in the realm of resource management. The Combinatorial Optimization branch is subdivided into two primary categories: exact methods and approximate methods. Within the approximate methods, two classes are identified: heuristics and meta-heuristics. The meta-heuristics category is further classified into population-based, single solution-based, and hybrid meta-heuristics.

In the ML domain, three main categories are considered: Dataset-based techniques (including supervised learning, unsupervised learning, and deep learning), Action/Reward-based techniques (such as reinforcement learning and deep reinforcement learning), and hybrid ML techniques that integrate various ML models.

Finally, hybrid methods refer to the combination of Combinatorial Optimization and ML techniques, where both approaches are utilized synergistically to optimize resource management.

In the following subsections, we review the Combinatorial Optimization and ML techniques used in the literature.

A. COMBINATORIAL OPTIMIZATION

Numerous studies have been conducted on resource management within the Edge, Fog, and Cloud environments, employing Combinatorial Optimization techniques. Initially,

we provide an overview of the exact methods utilized in these studies. Following that, we outline the approaches that have opted for approximate methods.

1) EXACT METHODS

The exact techniques aim to find the optimal solution and assesses its optimality [116]. They are often used for resource management problems with small to moderate complexity where exhaustive or systematic exploration is feasible. Common exact techniques include Integer Linear Programming (ILP), Mixed Integer Programming (MIP), and Constraint Programming (CP). However, due to their high computational cost, their applicability becomes limited in large-scale or highly dynamic Edge-Fog-Cloud environments. For instance, [101] proposed an approach utilizing an ILP model for placing IoT application modules within Cloud-Fog environments. The primary objective is to minimize latency between nodes during module placement. Similarly, [102] introduced an approach employing the same technique to optimize IoT application deployment in Fog-Cloud environments. The aim is to identify suitable Fog-Cloud nodes for each application module while ensuring adherence to QoS and security constraints. Reference [87] addressed the problem of assigning IoT tasks to Cloud-Fog resources to reduce total execution time and cost. They utilized a MILP to achieve a globally optimal solution.

2) APPROXIMATE METHODS

The approximate methods are widely used in the literature due to their ability to provide efficiently approximate near-to-optimal solutions, particularly to complex problems. They are mainly divided into two classes: heuristics and meta-heuristics.

- 1) **Heuristics:** In computer science, heuristics, as elucidated by Peres et al. [11], refer to the application of practical principles and exploratory search methods designed to address complex problems where obtaining an exact solution within a feasible timeframe is impractical due to high computational complexity. This approach often involves a degree of uncertainty, resulting in non-deterministic outcomes. For example, [103] proposed the Module Mapping (MM) heuristic, which allocates modules to the nearest Fog resource with adequate computational and storage capabilities. If no suitable Fog node is available, the module is automatically offloaded to the Cloud. Similarly, [101] introduced the Latency Aware Placement Heuristic (LAMPH), which prioritizes the placement of time-sensitive modules on nearby Fog resources to minimize inter-node latency. The authors in [68] presented a Greedy heuristic for resource management, aiming to maximize the number of served end-users while adhering to a predefined delay threshold. Initially, the problem is formulated using ILP, but due to its high time complexity, a low-complexity Greedy heuristic is employed. In another

TABLE 3. Classification based on resource management dimensions.

Dimensions		Works
Element	Monolithic Application	[36], [37], [18], [68], [69]
	Independent tasks	[38], [70], [71], [60], [72], [73], [74], [75], [16], [76], [40], [77], [78], [17], [41], [79], [80], [81], [82], [6], [83], [61], [42], [84], [39], [47], [85], [86], [87], [88], [89], [90], [91], [92], [93], [94], [95], [96], [97], [98], [99], [100]
	Dependent tasks	[101], [102], [103], [104], [105], [106], [44], [43], [107], [108], [7], [109], [8], [45], [110], [111], [112], [113], [49], [114], [115]
	Data	[51], [52], [53]
Aspect	Placement/ Resource Allocation	[101], [102], [87], [103], [68], [104], [105], [106], [38], [51], [107], [74], [108], [7], [109], [16], [76], [77], [78], [17], [111], [92], [96]
	Scheduling	[70], [71], [43], [60], [72], [73], [75], [45], [47], [53], [89], [112], [113], [97], [114], [115]
	Offloading	Partial: [40], [8], [41], [42], [84], [39], [90], [61], [91], [93], [94], [98], [99], [100] Full: [52], [83], [69]
	Migration	[88], [44], [85], [79], [80], [110], [86], [81], [6], [82], [18], [36], [37], [95], [49]
Environment	Single	Edge Computing: [88], [85], [41], [79], [93] Mobile Edge Computing: [80], [82], [92], [18], [83], [36], [37], [42], [84], [112], [91], [94] Fog Computing: [68], [44], [107], [60], [52], [73], [76], [78], [81], [61], [97], [115] Cloud Computing: [104], [75], [47], [114]
	Hybrid (Multiple)	MEC/Edge-Cloud Computing: [38], [7], [17], [110], [6], [90], [109], [95], [113], [98], [99], [100] Fog-Cloud Computing: [101], [102], [87], [103], [105], [106], [70], [71], [43], [51], [72], [74], [108], [16], [40], [77], [8], [45], [86], [111], [53], [89], [49], [96], [69] Edge-Fog-Cloud Computing: [39]

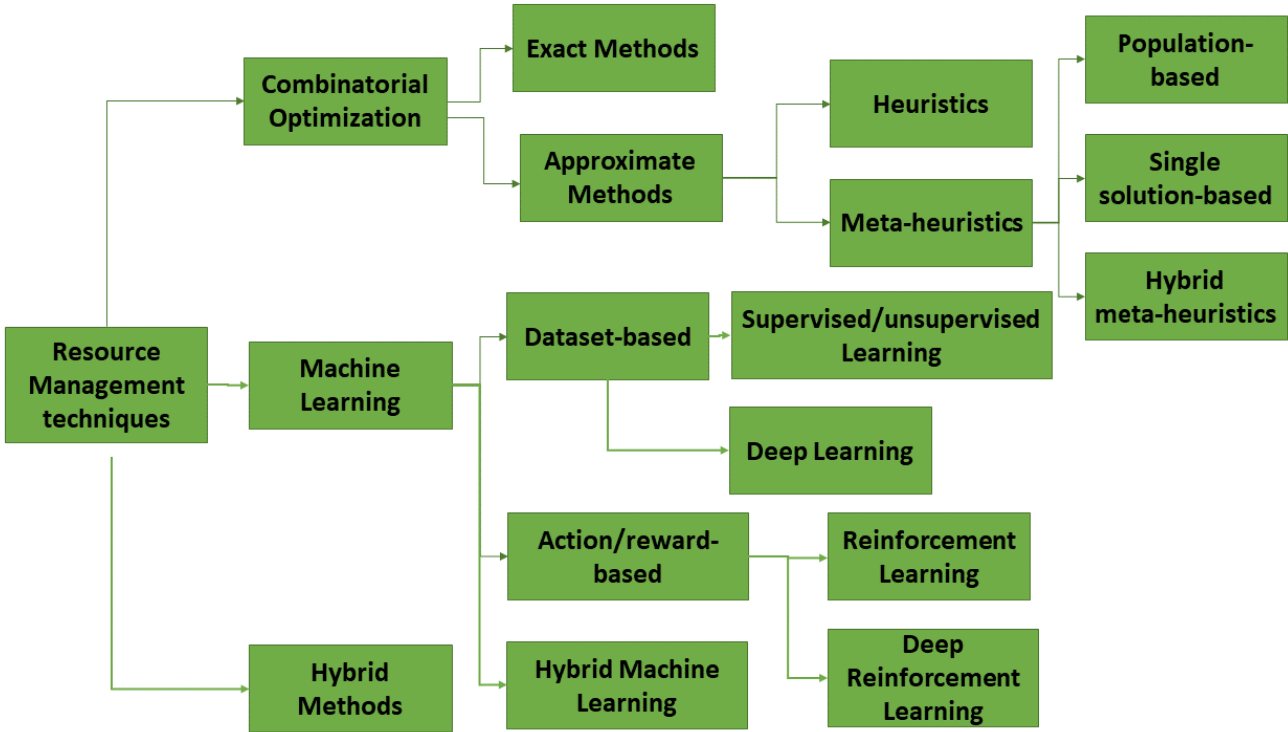


FIGURE 7. Resource management techniques.

study, [104] developed two heuristics to identify sub-optimal deployment configurations for Business Process tasks, optimizing cost, security, and risk thresholds. Reference [105] proposed the Latency-Aware Module Placement (LAMP) heuristic, which organizes IoT application modules based on execution order to optimize their placement across Fog-Cloud resources. Reference [106]

introduced the Most Delay-sensitive Application First (MDAF) heuristic, prioritizing the placement of delay-sensitive services on the nearest Fog resources to reduce overall latency. Reference [38] designed a deterministic Greedy algorithm to allocate application components to cost-effective Cloud-Edge offerings while satisfying security constraints. Reference [44] presented the Mobility

Greedy Heuristic (MGH), which sequentially places IoT services within a Fog architecture to minimize transfer time between newly placed services and previously assigned ones. Reference [70] proposed two heuristic-based algorithms for scheduling IoT tasks on voluntary Fog-Cloud resources, focusing on cost minimization and deadline adherence. Reference [71] developed the PSG and PSG-M semi-greedy algorithms for efficient IoT task mapping, emphasizing high QoS and energy efficiency. Reference [88] addressed the reduction of virtual machine migrations in Edge Computing through a heuristic-based migration scheme. Most recently, [96] introduced a heuristic approach for service placement that dynamically adjusts the number of service replicas based on current workload and available resources, aiming to minimize costs and reduce delays for delay-sensitive services.

- 2) **Meta-Heuristics:** A Meta-heuristic is an advanced iterative main process that orchestrates and refines the operations of subordinate heuristics to effectively generate high-quality solutions [12]. These techniques are broadly categorized into three distinct classes. The first class encompasses population-based meta-heuristics, which leverage a collection of potential solutions to explore the search space. The second class comprises single solution-based meta-heuristics, which focus on iteratively improving a single candidate solution. The third class integrates techniques from both the population-based and single solution-based approaches, combining their strengths to enhance solution quality and efficiency.

a) Population-based Meta-Heuristics: In this category, an initial population of multiple solutions is generated, and in each subsequent iteration, a subset of these solutions is systematically manipulated to explore and identify regions of the search space that yield superior solutions [117]. In each iteration, a subset of the population is selected and modified according to algorithm-specific operators (e.g., velocity and position updates in particle swarm optimization, crossover and mutation in genetic Algorithms), guiding the search toward promising regions. Population-based meta-heuristics are extensively utilized in addressing resource management challenges due to their ability to effectively manage complex, large-scale optimization problems and their proficiency in discovering near-optimal solutions. For instance, Particle Swarm Optimization (PSO) code solution as particle with position and velocity vectors. Particles update their velocity based on their personal best and the global best positions, guiding the swarm toward high-quality allocations. Reference [43] proposed an enhanced particle swarm optimization (IPSO) algorithm designed to optimize the placement of workflow tasks across Fog-Cloud resources, achieving a balance between makespan and cost. Similarly, [68] developed, alongside a Greedy heuristic, a PSO strategy that delivers near-optimal performance within a limited number of iterations. Genetic

Algorithms (GAs) have also been widely applied in this domain. In contrast to PSO, GAs represent solutions as chromosomes and evolve them through processes like selection, crossover, and mutation. Reference [51] introduced a GA-based approach for placing data generated by smart city sensors on Fog nodes, aiming to reduce latency and improve processing efficiency. Reference [107] presented a multi-objective genetic algorithm (MAPO) for application placement in Fog environments, focusing on minimizing compilation time, cost, and energy consumption. Additionally, [72] proposed a cost-aware genetic-based algorithm (CAG) to optimize task placement, reducing overall processing costs while enhancing task completion times. Reference [44] developed a Mobility-aware Genetic Algorithm (MGA) for service placement in Fog environments, specifically addressing the mobility of IoT devices. Other optimization techniques have also been explored. Reference [60] employed a Moth Flame Optimization (MFO) algorithm for task placement in Fog nodes, ensuring higher QoS for CPS applications. We note that MFO algorithm is inspired by the navigation behavior of moths around light sources, where candidate solutions (moths) update their positions by spiraling around flame positions. Ant Colony Optimization (ACO) is a widely used population-based metaheuristic inspired by ant foraging. ACO constructs solutions via artificial ants depositing pheromones on promising paths. Reference [52] proposed two meta-heuristics utilizing ACO and PSO to allocate sensor-generated data to Fog resources, aiming to minimize global response times. Reference [113] developed a workflow scheduling strategy based on ACO to improve the distribution of applications between Edge and Cloud servers, with a focus on computational efficiency. Several studies have emphasized multi-objective optimization. Reference [73] introduced an evolutionary adaptive multi-objective optimization task scheduling method (Fog-AMOSM) for CPS, targeting the simultaneous minimization of time and cost. Reference [74] presented a conceptual framework for Fog-Cloud control middleware, leveraging PSO to enhance resource utilization while reducing latency and costs. Furthermore, [108] proposed a Deadline-oriented Service Placement (DoSP) algorithm, which analyzes service placement response times to optimize the deployment of workflow-based IoT applications. Recent advancements have also explored innovative metaheuristic techniques such as Firework Optimization (FWO), Harris Hawk Optimization (HHO), Grey Wolf Optimization (GWO), and Whale Optimization Algorithm (WOA). FWO is inspired by the explosion of fireworks, where each solution (firework) generates sparks (new candidate solutions) based on its fitness; the fitter the solution, the more sparks it produces, thus balancing exploration and exploitation of the search space [112]. For instance, a Blockchain-based Scheduling-based Fireworks Algorithm (BSFW) was proposed to enhance security while optimizing energy

consumption and makespan. GWO, which emulates the social hierarchy and hunting strategy of grey wolves, has been used to solve complex scheduling problems. In [97], a multi-objective, cost-aware discrete GWO (MoDGWO) algorithm was developed for scheduling Bag-of-Tasks (BoT) in IoT applications within Fog environments, optimizing execution time, cost, and reliability. HHO draws inspiration from the surprise pounce strategy of Harris hawks during hunting, offering efficient exploration and exploitation capabilities. As demonstrated in [114], an Enhanced HHO-based scheduler was introduced for dynamic workflow scheduling in cloud computing, accounting for task dependencies. Finally, WOA, which imitates the bubble-net hunting technique of humpback whales, has been used to solve offloading challenges in dynamic edge computing environments. In [99], WOA was hybridized with GWO to improve QoS by minimizing latency and enhancing decision-making in uncertain and dynamic IoT scenarios.

b) Single solution-based Meta-Heuristics: These methods are commonly referred to as trajectory methods. In contrast to population-based meta-heuristics, trajectory methods begin with a single initial solution and iteratively deviate from it, tracing a path through the search space. Some trajectory methods are considered advanced extensions of local search algorithms [118]. Tabu Search (TS) and Simulated Annealing (SA) are widely used. TS uses memory structures to avoid revisiting solutions, promoting exploration, while SA accepts worse solutions with a probability that decreases over time, helping to escape local minima. In [7], the authors proposed a TS-based approach to optimize the placement of service-chained Virtualized Network Functions in MEC-Cloud environments, aiming to minimize end-to-end delays and deployment costs by systematically exploring the search space and avoiding revisited solutions.

c) Hybrid Meta-Heuristics: The approaches in this category utilize both population-based and single-based meta-heuristics. However, it is important to note that there have been relatively few studies conducted within this specific category. For instance, [109] introduced the IoT-Data-Intensive Service Components (IDISC) approach for deploying IoT services in Edge Cloud-Hybrid Environments. These services often exhibit complex dependencies, including data and logical dependencies. To reduce data transmission time, the authors proposed an optimization algorithm combining ACO and SA. More recently, in [85], researchers combined GA and SA for Edge Server Selection (SS), aiming to minimize the overhead experienced by mobile users in terms of time and energy consumption.

B. MACHINE LEARNING

In this subsection, we outline the approaches employing ML-based techniques to address resource management

challenges. It's worth highlighting the extensive adoption of ML in recent years. We can categorize these techniques into three main groups: Dataset-based techniques, Reward/Action-based techniques, and Hybrid ML techniques.

1) DATASET-BASED TECHNIQUES

This category regroups the approaches that use ML techniques belonging to supervised, unsupervised, and deep learning fields. These techniques exploit real-world or synthetic datasets to build intelligent models capable of making decisions.

- 1) **Supervised/Unsupervised Learning:** Supervised learning enables algorithms to generalize from labeled data to predict outcomes for new, unseen cases. Common techniques include regression models, which predict continuous values such as execution time or energy consumption, and classification models, which assign categorical labels like the optimal resource type. In [16], the authors developed a Gaussian Process Regression (GPR) algorithm for Fog-Cloud Allocation (GPRFCA), using regression to allocate delay-sensitive tasks to Fog resources and tasks with more flexible response times to the Cloud. Similarly, [78] proposed an energy-aware resource allocation method utilizing multiple linear regression to predict execution time and energy consumption for Fog-based applications. In another approach, [40] employed a Classification and Regression Tree Algorithm (CART) for module placement in IoT applications, selecting the most suitable Fog device for each module through a combination of classification and regression techniques. Furthermore, [77] introduced an AI-based task distribution algorithm (AITDA) that intelligently allocates tasks between Fog and Cloud servers to reduce response time and Internet traffic. In contrast, unsupervised learning involves identifying hidden patterns in unlabeled data through techniques such as clustering, which groups similar resources or tasks based on their characteristics [13]. For instance, [76] addressed the challenge of smart farming by using clustering to group resource-efficient Fog nodes and map tasks to the appropriate clusters based on resource availability.
- 2) **Deep Learning:** According to [14], deep learning is a class of machine learning algorithms that utilize multiple layers of nonlinear processing units for feature extraction and transformation. Various techniques fall under this category. For example, Deep Neural Networks (DNNs) are known for their robust function approximation capabilities, which are useful for complex decision-making problems. In [8], the authors developed the DNNECTS algorithm, which uses DNNs to make optimal partitioning decisions for workflow applications. This approach considers both execution time and resource availability to effectively allocate tasks between local devices and

Fog nodes, followed by intelligent task scheduling. Deep Supervised Learning (DSL), which combines the representational power of deep networks with labeled training data, is another key technique for solving classification and regression problems. In [91], the DSLO framework demonstrated DSL by learning to map dynamic workload characteristics to optimal computation offloading decisions, adapting to time-varying conditions through continuous updates. Additionally, Long Short-Term Memory (LSTM) networks, a specialized type of Recurrent Neural Networks (RNNs), have shown strong capabilities in modeling temporal sequences and time-series data. For instance, in [91], LSTM was employed to capture daily resource usage patterns of Edge service providers. This predictive modeling enabled accurate bandwidth traffic forecasts, which were subsequently utilized to optimize dynamic resource allocation within Edge computing environments.

2) ACTION/REWARD BASED TECHNIQUES

In contrast to the preceding category, the techniques within this branch empower an Artificial Intelligence system to acquire decision-making capabilities through rewards for its actions. This category primarily encompasses techniques such as reinforcement learning and deep reinforcement learning.

1) **Reinforcement Learning:** Reinforcement learning is a domain within ML that explores problems and their respective solutions, where agents learn optimal actions through interaction with their environment to maximize a specific numerical reward [15]. Several studies have employed the Q-Learning technique, a model-free RL method that iteratively updates a Q-table to estimate the value of state-action pairs. This technique has been used to optimize task offloading, scheduling, and service placement in Edge and Fog computing environments. For instance, the authors in [41] proposed a Q-learning-based algorithm to achieve near-optimal task offloading decisions, aiming to minimize both energy consumption and execution latency. Similarly, [45] developed a RL-based task scheduling policy designed to reduce makespan and communication costs in real-time environments. In the context of dynamic service placement, [79] introduced an RL-Dynamic framework that optimally places services on Edge servers while accounting for vehicle mobility and varying service requests. Recently, [89] proposed an RL-based Fog scheduling algorithm (RLFS), which efficiently handles real-time requests sensitive to delays by assigning them to Fog resources, whereas non-real-time requests are offloaded to the Cloud layer.

2) **Deep Reinforcement Learning:** It is a subfield of ML that combines reinforcement learning and deep learning. The distinct property is that the agent tries to learn trial and error from feedback that is simultaneously sequential, evaluative, and sampled by leveraging powerful non-linear function approximation using a deep neural network

[119]. Several studies have utilized Deep Q-Networks (DQN), a model-free reinforcement learning technique that uses deep learning to approximate the Q-value function for decision-making. In task migration, [80] used DQN to enable a centralized controller to decide when and where tasks should migrate based on user mobility. Similarly, [86] employed DQN for determining the optimal number of services to offload between the Cloud and Fog, considering fluctuating network conditions. In [92], DQN was used for real-time service placement decisions at the Edge, optimizing service pre-positioning for incoming tasks. Additionally, [49] developed a DDQN-based migration strategy to minimize latency and energy consumption in heterogeneous environments. Deep Deterministic Policy Gradient (DDPG), another popular reinforcement learning technique, is used for continuous action spaces and policy optimization. In [81], DDPG was applied in the OctoFog framework to efficiently migrate services in smart city applications, while [82] proposed a Deep Recurrent Actor-Critic based service migration method (DRACM) to optimize decision-making under incomplete information. Multiple Buffer DDPG (MB-DDPG), a variant of DDPG, was introduced by [110] to optimize IoT microservice deployment in Edge-Cloud environments, considering dynamic environmental characteristics. Multi-Agent Deep Deterministic Policy Gradient (MADDPG), an extension of DDPG for multi-agent environments, was used in [94] to explore suboptimal offloading strategies for vehicles. Counterfactual Multi-Agent Deep Reinforcement Learning (COMA), another multi-agent approach, was adopted by [18] to address task completion under mobility and energy constraints. Lastly, Rainbow DQN, an enhanced version of DQN, was applied by [93] to minimize both delay and energy consumption for task offloading in vehicular networks.

3) HYBRID MACHINE LEARNING

This category integrates previously presented ML techniques. In contrast to other ML sub-categories, this one remains relatively less explored in the resource management field. Typically, dataset-based techniques are used to forecast future states of the network, Fog/Edge nodes, or user mobility. Subsequently, Action/Reward techniques are leveraged to discover optimal or near-optimal allocation solutions [29]. For instance, [42] proposed an Online Predictive Offloading (OPO) algorithm for addressing the task offloading problem in Edge environments. Their approach employs LSTM networks to predict the feature information of incoming tasks and the load of Edge servers in the next time slot. The predicted information is then utilized in a reinforcement learning decision model, specifically using DQN, DDQN, and Dueling DQN, to formulate an offloading scheme for the anticipated task. Similarly, [84] developed an adaptive task offloading decision algorithm that combines DDQN with k-nearest neighbors (KNN) to continuously learn

experience values, thereby generating offloading policies for multiple tasks based on the current channel state. Moreover, [95] investigated the joint optimization of dynamic service migration and resource allocation within MEC. Their methodology incorporates LSTM to forecast IoT users' mobility behaviors, which enhances migration accuracy and reduces the action space. The Parameterized Deep Q-Network (PDQN) algorithm is then employed to tackle the challenges posed by the continuous-discrete hybrid action space during the optimization process, all while preserving the original structure of the action space.

C. HYBRID METHODS

This category combines Combinatorial Optimization with ML methods. We note that only a few works have been proposed in this context. For instance, [39] developed a heuristic that employs a profit-based mechanism to determine the execution of applications across various nodes (Edge, Fog, or Cloud). This approach ensures that the node with the highest profit manages the deadline-aware execution of tasks, utilizing reinforcement learning for task assignment based on the heuristic's outputs to meet the objective function effectively. Similarly, [111] introduced the Cost and Reliability-aware Eagle-Whale (CREW) approach, which first predicts service execution times using the Extreme Gradient Boosting (XGBOOST) regression model. They subsequently integrated the Whale Optimization Algorithm (WOA) with Eagle optimization to identify optimal placement configurations. Recently, [47] combined Binary Particle Swarm Optimization (BPSO) with Artificial Neural Networks (ANN) to enhance workload scheduling and balancing in Cloud Computing.

Table 6 provides a concise overview of the studies mentioned, comparing them based on five key properties. The *approach category* is classified as Exact (EM), Approximate (AM), Dataset-based (DB), Action/Reward-based (ARB), or Hybrid method (HM). The *algorithm type* includes categories such as Linear Programming (LP), Heuristic (H), Meta-Heuristic (MH), Hybrid Meta-Heuristic (HMH), supervised learning (SL), unsupervised learning (USL), deep learning (DL), reinforcement learning (RL), deep reinforcement learning (DRL), and hybrid machine learning (HML). Additionally, the *model-based algorithm* specifies the type of model used, including integer linear programming (ILP), mixed-integer linear programming (MILP), Genetic Algorithm (GA), and Simulated Annealing (SA). The *tool-languages* and the *application domain* are also highlighted. Finally, the *Experimental Evaluation* section outlines the data types used, including Synthetic (S), Real (R), and Mixed datasets (S-R), along with details on the maximum number of resources (MR), applications (MA), workflows (MW), business processes (MBP), offers (MO), services (MS), modules (MM), tasks (MT), as well as the number of IoT devices (MIoT) and sensors (MSN) involved in data generation.

V. QoS METRICS AND ENVIRONMENT-AWARE RESOURCE MANAGEMENT

QoS parameters have particularly received significant attention within the resource management community, in both static and dynamic environments. This section explores and classifies the existing approaches based on the optimized QoS metrics and type of environment.

A. OPTIMIZED QoS METRICS

Typically, the primary goal of QoS-based resource management approaches is to ensure and maintain targeted QoS parameters. A comprehensive taxonomy, depicted in Figure 8, illustrates the different QoS parameters handled in the literature. Mainly, we distinguish four categories: *performance*, *cost*, *energy consumption*, and *security*.

The primary objective of performance metrics is to optimize time constraints in different processes. We classify them into two main categories: *global metrics* and *composing metrics*. Global metrics encompass a comprehensive set of performance indicators that take into account various aspects of the system or process, such as response time. They provide a comprehensive and holistic view of the overall performance, taking into account the collective impact of different composing metrics. On the other hand, composing metrics focus on atomic metrics that provide specific and detailed insights into the performance of individual aspects of the system, for instance, the execution time. Additionally, cost metrics refer to the total cost associated with processing, storage, and communication resources required for the execution of applications [102]. They refer to financial implications and resource utilization efficiency. On the other hand, energy consumption represents the amount of energy consumed during the execution of a system [44], [81]. In the context of resource management, security constraints refer to a set of cryptography algorithms, measures, and controls that are used to ensure the appropriate mapping of services to secure nodes based on the specific security requirements of the services [40], [102].

- 1) **Performance:** Time-related constraints have been tackled in the literature to optimize the performance metrics [38], [85], [106]. The primary objective of these studies is to minimize the overall latency, delay, or response time, which encompasses the total elapsed time from the moment a request is sent to when a suitable response is received [109]. Additionally, various approaches have been proposed to minimize the makespan, defined as the duration from the initiation of the first task to the completion of the last task [43], [45], [87], [107]. This measure is also referred to as completion time or timespan. Within the context of time optimization, latency and makespan may comprise several underlying metrics, including execution (ET), communication (CT), transmission (TT), switching (SWT), deployment (DT), migration (MT), queuing (QT), propagation (PT), and serving time (ST). The processing latency, also known

as execution time, or computing delay, quantifies the duration required to complete the execution of a task on a computing node. Notably, considerable research efforts have been dedicated to investigating and enhancing this metric [36], [83]. Communication latency, alternatively labeled as transfer time, focuses on minimizing the time taken for data transmission between nodes in network [36], [83]. In the case of offloading approaches, the transmission time, commonly known as offloading time, represents the expected duration for transmitting a task from an end-user device to a computing node. Reference [92] specifically aimed to minimize the switching latency, which denotes the time required to transmit a service from the Cloud to the Edge. Furthermore, certain studies [68] have incorporated the deployment delay, where the delay model accounts for the cumulative time needed to deploy a container, including downloading and installing the application on Fog nodes. The time incurred during the migration of services or tasks between nodes has garnered significant attention in works that deal with migration problems [18], [36], [88]. Moreover, queuing time, which reflects the period in which a task waits to be processed in the queue of a device, has been incorporated into the time models of many works, such as [7]. The propagation delay, which represents the time it takes for a single bit to cross from one end of a link to the other, has also been considered in transmission models within papers [71]. Finally, the studies conducted by [44] and [106] seek to minimize delay or deadline violations. These violations occur when the effective response time of a request exceeds the maximum response time tolerated.

- 2) **Cost:** Several works seek to minimize the execution cost (EC) of tasks on Edge-Fog-Cloud resources [87], [102], [104]. It is mainly referred to in other works as the summation of CPU usage cost and memory usage cost (CPUC and RAMC). Additionally, in certain studies, the EC is calculated based on the machine's usage tariff per unit time (MTC). The communication cost (CC) (Bandwidth Usage, Network cost, or transfer cost) incurred for transferring data in the network between end-device and nodes or inter-nodes has been investigated in [38], [102], and [104]. Transmission cost (TC) occurs for transmitting tasks from end-device to computing nodes is mainly addressed by offloading approaches [40]. Additionally, the cost incurred for storing data (SC) is considered in [40] and [86]. The authors in [73] aim to minimize the service level objective (SLO) violation penalty due to service timeout (SLOVP). Furthermore, [70] considered the minimization of the deadline violation cost (DVP), which refers to the penalty that the Fog Broker must pay for the delay violation. Finally, many papers worked on the optimization of resource utilization (RU) [7], [79] by avoiding the over-utilization of the resources.
- 3) **Energy consumption:** Five aspects of energy have been investigated in the literature. The first one is related to the

energy generated when a task is executed by computing resources (EEC) [76], [107]. The communication energy consumption (CEC) that is incurred when data generated by the end devices or computing nodes is transmitted through a network has been also studied in [44] and [107]. In addition, energy consumption generated when tasks are transmitted from end devices to nodes has been addressed in offloading strategies (TEC) [8], [40], [41]. Efforts have also been made to reduce energy consumption when tasks or services are migrated from one node to another (MEC) [18], [36]. Lastly, very few works considered the waiting energy consumption generated when tasks are waiting in queue for execution (WEC) [42].

- 4) **Security:** Several works have been proposed to handle security-related issues. In [38], the authors aim to satisfy the security controls constraints along with associated implementation levels that characterize each application component. In [102], the authors assigned a security rating to Fog nodes and based on the security level requirement attached to the module, the placement is performed. Whereas, in [40], the authors expressed security as a set of parameters, which are authentication, confidentiality, integrity, and availability. Each parameter is measured by the performance of its security algorithms implemented in each Edge node. Finally, privacy violation risk is treated in [104]. The authors considered that the maximization of sensitive task security refers to the minimization of the risk level to which it is exposed on all paths linking it to its complementing task. In, [100] explores critical aspects such as privacy and protection against repudiation attacks by leveraging Blockchain and smart contracts in multi-user and multi-server MEC-enabled 5G and wireless networks.

Several other metrics have been explored in the literature, including the fraction of tasks completed within the deadline [72], the number of requests exchanged within a network [76], system reliability [97], [111], [115], throughput [74], [75], the failure rate, which quantifies how often tasks fail due to resource mismatches or scheduling issues [114], and fault tolerance, which reflects the system's capability to operate correctly even in the presence of faults or failures [115].

It is important to note that, in many real-world scenarios, QoS metrics do not operate in isolation. Optimizing one often leads to the degradation of another. For instance, minimizing latency may lead to increased energy consumption due to the use of higher-performing (and more power-hungry) resources such as Cloud [49], [83], [107]. Similarly, enforcing high security may increase transmission cost or processing delay due to encryption and verification overheads [38], [40]. Studies such as [73] and [107] have addressed this by applying multi-objective optimization techniques, where performance trade-offs are resolved using Pareto-based approaches, weighted functions [18], [44], [83], or heuristic strategies [104]. Understanding these conflicts is essential for designing adaptive and context-aware resource management

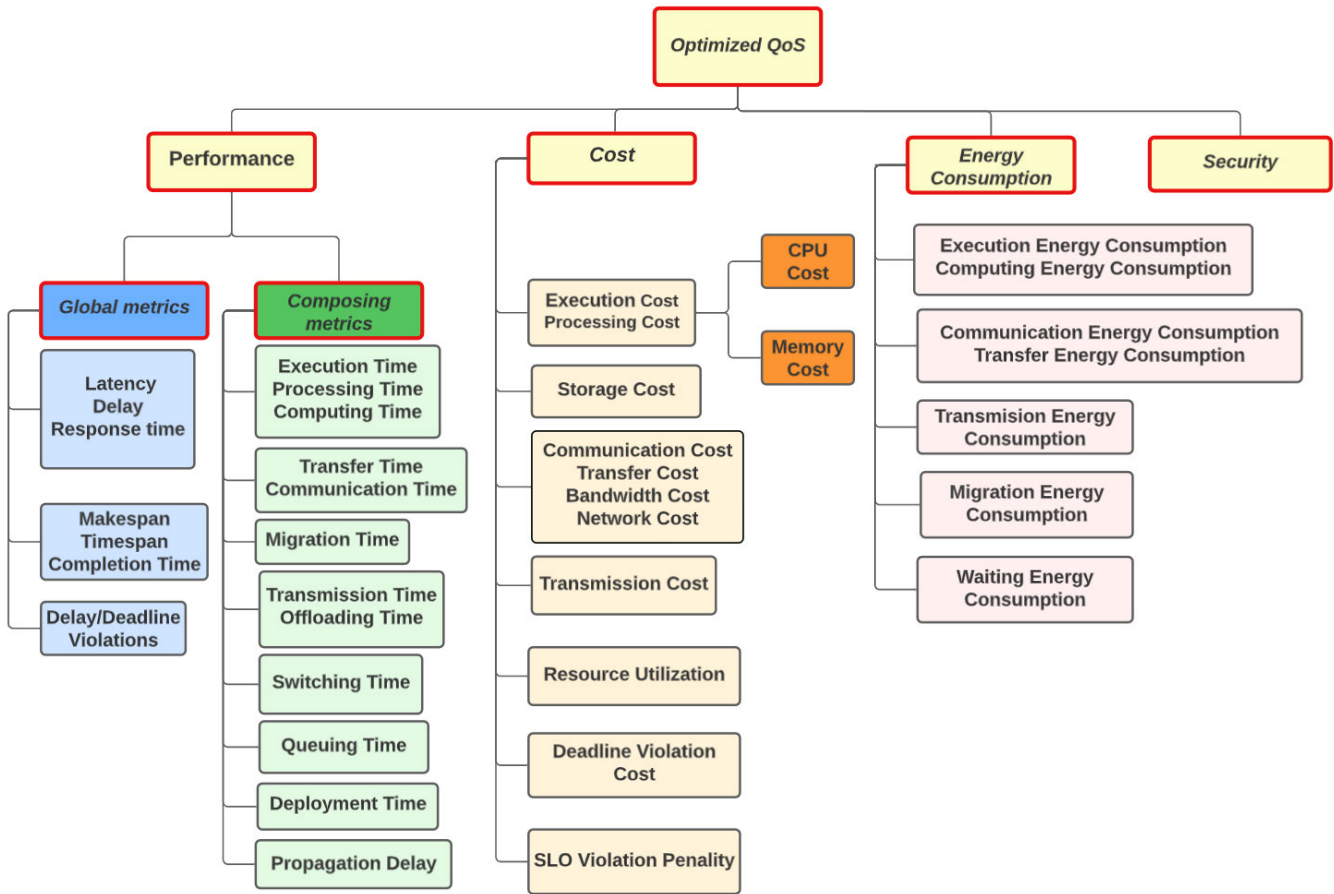


FIGURE 8. QoS taxonomy of resource management.

algorithms that can balance conflicting objectives according to application requirements.

Table 4 presents the common formulas used for calculating the different QoS metric.

Table 5 presents a comparative analysis of the most commonly optimized QoS parameters including performance, cost, energy consumption, and security in resource management studies, categorized by the employed techniques. Table 7 provides a more detailed breakdown of these metrics. It is important to note that the comparison primarily focuses on the QoS metrics explicitly optimized through the objective functions in the reviewed works.

B. RESOURCE MANAGEMENT ENVIRONMENT TYPES

As illustrated in Figure 9, the studied works mainly consider two types of IoT environments: *static* and *dynamic*. The nature of these environments is related to Edge-Fog-Cloud nodes, network, application, and end-user features. Unlike the static environment where these parameters do not vary over time, in the dynamic one, they can change. In the literature, several works rely on the assumption that the targeted system is static [73], [87], [102]. This assumption is restrictive and limits the applicability of the proposed approaches. In fact, Fog-Edge nodes features (NF) like

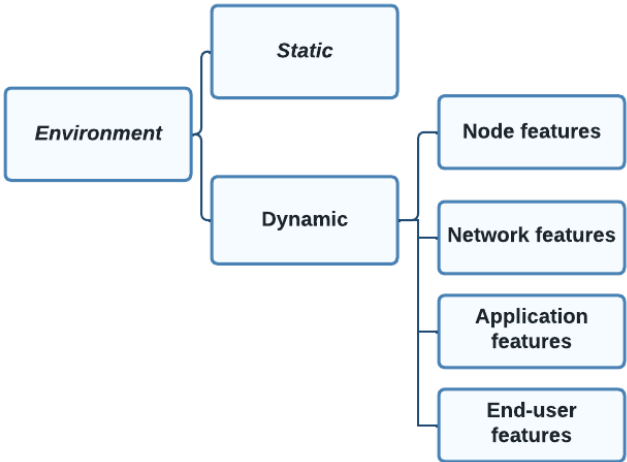


FIGURE 9. Resource management environments.

mobility, availability, and load may be dynamic. The Fog-Edge devices such as mobile phones or cars, can be on the move and change their locations in time. In addition, the availability state of the Fog computing platform changes dynamically due to several parameters such as on-off power switching, battery life limitations, and wireless access links

TABLE 4. QoS metrics and their formulas.

Metric	Formula	Description
Execution Time (ET)	1) $ET = \frac{NI}{MIPS}$ [29], [44], [60], [105] Where NI is the number of instructions of task with $MIPS$ computing capacity of the resource 2) $ET = \frac{L}{f}$ [84], [98] Where L is the required CPU cycles to complete a task with resource computation frequency capacity f (in cycles/sec).	Time taken to execute a task on computing node.
Communication Time (CT)	$CT = \frac{D_{size}}{Bandwidth/Datarate}$ [43], [49], [96], [112] Where D_{size} is the amount of data exchanged between nodes, and $Bandwidth$ is the communication link rate.	Time to transfer data D_{size} between nodes .
Transmission Time (TT)	$TT = \frac{task_{size}}{Bandwidth_{uplink}/Datarate}$ [49], [94], [99] Where $task_{size}$ is the size of the task's input data, and $Bandwidth_{uplink}$ is the uplink transmission rate.	Time required to transmit tasks from the end device to a computing node .
Switching Time (SWT)	$SWT = \frac{service_{size}}{Bandwidth_{cloud-edge}}$ [92] Where $service_{size}$ is the data size of the service and $Bandwidth_{cloud-edge}$ is the link rate between Cloud and Edge.	Time to switch services from Cloud to Edge.
Deployment Time (DT)	$DT = Time_{download} + Time_{install}$ [68], [108]	Time to deploy an application or container.
Migration Time (MT)	$MT = \frac{task_{size}}{bandwidth/Datarate}$ Where $task_{size}$ is the size of the entire task/container to migrate, and $Bandwidth$ is the migration link speed. [29]	Time to migrate task/service to another node.
Queuing Time (QT)	$QT = Time_{queue}$ [71], [74]	Waiting time experienced in the edge device or the scheduler queue.
Propagation Time (PT)	PT influenced by the distance between communicating nodes [7].	Time for a signal to travel through the medium.
Execution Cost (EC)	1) $EC = CPUC + RAMC$ where $CPUC$ and $RAMC$ represent the costs associated with CPU and memory usage, respectively. 2) $EC = MUP \times ET$ where MUP is the Monetary Unit Price per time unit, and ET is the Execution Time.	Represents the cost of resource utilization during task execution.
Communication Cost (CC)	$CC = D_{size} \times Transfercost$ where D_{size} is the size of the data and $Transfer cost$ is the cost per unit of data transferred	Cost for data transferred between nodes.
Transmission Cost (TC)	$TC = Task_{size} \times Cost_{bandwidth}$ [40]	Cost of sending a task from end-device to node.
Storage Cost (SC)	$SC = storage_{size} \times Storage cost$ [102], [104] where $storage_{size}$ is the amount of data to be stored and $Storage cost$ is the price per unit of storage	Cost associated with storing data on a computing node
SLO Violation Penalty (SLOVP)	$SLOVP = \sum_{i=1}^q [\gamma_i + \eta \times (t_i^{stop} - t_i^{deadline})]$ [73] where: q = number of SLO-violated requests, γ_i = fixed penalty for request i, η = penalty rate per unit delay, t_i^{stop} = actual response time, $t_i^{deadline}$ = service deadline.	Total penalty cost for violating SLOs for service timeout violations.
Deadline Violation Cost (DVC)	$V_i = \frac{\max(0, T_i^{response} - T_i^{deadline})}{T_i^{deadline}} \times 100$ $DVC = (V_i - (100 - T_i^{qos})) \times T_i^{penalty}$ [70] where: $T_i^{response}$ = response time of task i, $T_i^{deadline}$ = deadline time of task i, T_i^{qos} = QoS tolerance (percent), $T_i^{penalty}$ = penalty per % of violation.	Penalty cost when task deadline is violated.
Execution Energy (EEC)	$EEC = P \times ET$ [29], [44], [99], [107] Where P is computation power of node	Execution energy consumed during the processing of a task on a computing node
Communication Energy (CEC)	$EET = \gamma \times EC$ [44] Where γ is the energy cost per unit communication time between nodes.	Energy consumed for transmitting data between nodes.
Transmission Energy (TEC)	$TEC = \beta \times TT$ [49], [83], [93], [99] Where β is transmission power of end-user device.	Energy consumed for offloading tasks from end-device to the computing node.
Migration Energy (MEC)	$MEC = \alpha \times MT$ [49] Where α is transmission power of computing nodes Edge/Fog/Cloud.	Energy consumed by node for task migration over network.
Waiting Energy (WEC)	$WEC = WP \times QT$ [42] where WP is Waiting power of the terminal devices.	Energy consumed during task queuing time.

TABLE 5. Classification of optimized QoS metrics by technique.

QoS Metric	Combinatorial Optimization	Machine Learning	Hybrid methods
Performance	[101], [102], [87], [103], [68], [105], [106], [88], [38], [44], [71], [90], [96], [43], [51], [107], [60], [52], [73], [74], [108], [75], [7], [69], [112], [113], [97], [114], [109], [85], [99]	[16], [76], [77], [78], [8], [91], [41], [79], [89], [100], [45], [80], [110], [86], [81], [82], [6], [92], [18], [83], [36], [61], [37], [49], [93], [94], [98], [115], [42], [84], [95]	[39], [111], [47]
Cost	[102], [87], [103], [68], [104], [106], [38], [70], [53], [96], [43], [107], [72], [73], [74], [7], [97], [114]	[40], [17], [79], [45], [86]	[111]
Energy Consumption	[44], [71], [107], [112], [113], [85], [99]	[16], [76], [78], [17], [8], [91], [41], [89], [100], [81], [18], [83], [36], [61], [49], [93], [94], [98], [42], [84]	[39]
Security	[102], [104], [38]	[40], [100], [115]	

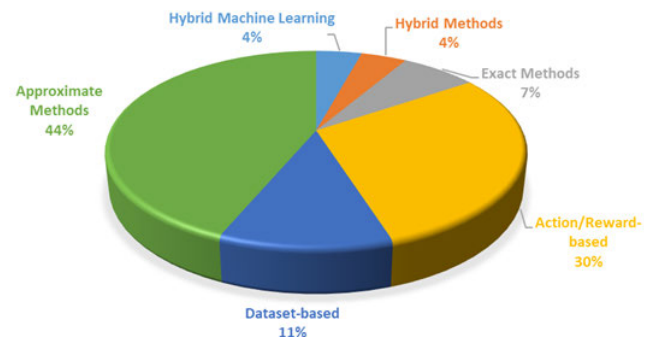
failures of Fog nodes [68]. Moreover, the number of user requests that the Edge/Fog node could handle may be time-varying. Many selected papers considered the dynamicity of Fog-Edge nodes [68], [110]. There are other dynamic features discussed in the literature related to IoT end-user (EUF) like the changing of user's location due to the user's mobility and continuous task generation [18], [36], [82]. In [6] and [79], the authors investigated another end-user feature when the number of requests for different types of services changes in each time slot. Additionally, network context (NWF) may also be dynamic. In [41], the authors addressed the time-varying of channel conditions between end-users and Edge servers in terms of channel gain. The change in network conditions (bandwidth) is discussed in [8]. Others as [7] discussed the dynamicity in the amount of data generated by the IoT layer and routed through the network. Finally, dynamicity is not only limited to changes in the deployment environment (end-user, network, and Edge/Fog nodes); application parameters (AF) may also be time-varying. Reference [17] addressed the variation in bandwidth traffic usage of Edge services providers. Reference [91] handled the dynamic change in the information of tasks through time in terms of workload and weight. Furthermore, application-level dynamics can also include real-time variations in task execution characteristics such as remaining duration, resource demand, or urgency. These variations require adaptive prioritization mechanisms [115], where priority scores are updated dynamically in response to the evolving state of each task, allowing the system to adapt to changes in workload patterns, execution time, and energy efficiency requirements.

Table 8 summarizes the considered environment nature associated with the considered features.

VI. DISCUSSION AND RESEARCH CHALLENGES

Building upon the study discussed above, this section aims to delve into a comprehensive statistical analysis that aligns with the questions posed in section II-A. *Q1: What are the different techniques of Combinatorial Optimization and Machine Learning used to optimize resource management? What is the technique the most used in the two fields?*

Figure 10 shows plainly the classification based on the category of techniques summarized previously in Table 6. Out of the 71 reviewed papers, we notice that the approximate methods have the highest percentage of used techniques by 44%, while Action/Reward-based techniques ranked in second place with 30%.

**FIGURE 10.** Classification based on the main categories of techniques.

Furthermore, Figure 11 effectively captures the classification based on the family of techniques. Starting with the approximate methods, it is shown clearly that the heuristic technique and population-based meta-heuristics are the most widely used with 24% and 20%, respectively. Moreover, we can see clearly that supervised, unsupervised, DL, and RL techniques are less used compared to DRL techniques 20%. Besides, we can clearly note from Figure 12 that in addition to heuristics, genetic Algorithm-based meta-heuristic, and Particle Swarm Optimization-based meta-heuristic are the most adopted due to their efficiency in finding near-optimal solutions for complex problems. Furthermore, we can notice that the technique of ML Deep Q-Network is well-considered in research papers. In summary, we can conclude from the discussed approaches that considerable attention has been directed toward approximate methods for addressing resource management problems. This is due to their simplicity and efficiency in finding solutions close to optimal, especially when dealing with large-scale systems. However, it is worth noting that approximate methods face limitations in handling dynamic environments, which has led to a recent shift in interest toward DRL techniques. DRL-based algorithms have

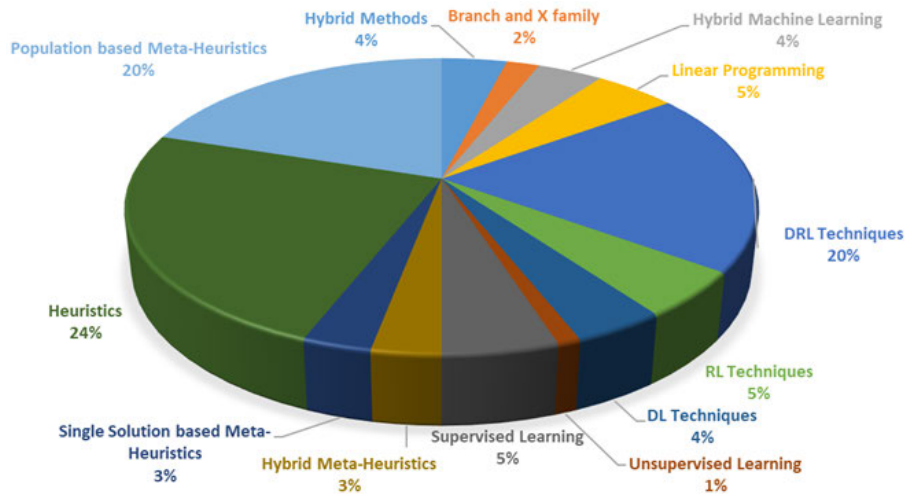


FIGURE 11. Classification based on the families of categories.

shown very interesting abilities in learning optimal solutions and their efficient adaptability, making them especially well-suited for dynamic environments.

Q2: Are there specific resource management aspects where Combinatorial Optimization techniques are more commonly employed compared to ML?

Figure 13 classifies resource management techniques. Combinatorial Optimization methods are mainly used in Resource Allocation/Placement and Scheduling, excelling at optimizing resource allocation, task assignment, infrastructure use, and operation scheduling to maximize QoS. ML-based techniques are primarily applied to offloading and migration problems. These involve transferring tasks from end devices to Edge/Fog/Cloud resources and moving tasks between systems, benefiting from the learning and adaptability capabilities of ML models

Q3: Are Combinatorial Optimization techniques more commonly utilized than ML techniques for managing specific element in resource management? Figure 14 suggests that ML-based resource management approaches primarily target optimizing independent tasks, which can be executed without dependencies on other tasks. This focus is often associated with ML-based approaches' concentration on migration or offloading problems, as discussed in Q2. Managing workflow migration, on the other hand, presents additional challenges and complexities, as noted in [29]. In contrast, Combinatorial Optimization approaches encompass a broader scope by not only addressing independent tasks but also tackling workflow resource management problems. These problems involve managing interconnected tasks, necessitating coordination and scheduling to ensure efficient resource allocation. This broader applicability of Combinatorial Optimization techniques makes them well-suited for handling diverse challenges, including those related to workflows and task dependencies.

Q4: What are the diverse QoS metrics prioritized and optimized in the studied papers? Are there particular QoS

metrics where ML techniques are more frequently utilized for optimization compared to Combinatorial Optimization? In the literature, a significant focus has been made to handle performance-related metrics, rendering them the most extensively studied QoS parameters. This is attributed to the primary objective of resource management approaches to effectively meet the requirements of time-sensitive applications in terms of delay and latency. Among the considered metrics, we can see in Figure 15 that execution (42%), communication (19%), and transmission (19%) delays are the most prominently studied factors in performance-based approaches. Following closely, cost and energy parameters have also been widely considered. As depicted in Figure 16, compared to other cost metrics, execution, and transfer costs are the most studied metrics with 38% and 26%, respectively. However, energy is less addressed in literature than cost. We assume that the majority of existing resource management approaches seek to maximize the satisfaction of users' requirements and experiences. Figure 17 clearly demonstrates that execution and transmission energy consumption are the most considered with 44% and 27%, respectively. These metrics are primarily tackled through offloading approaches to mitigate the energy generated during execution and transmission tasks within the network. On the other hand, Figure 18 reveals that various security aspects have been explored to enhance resource management security. However, it is worth noting that the number of studies focusing on security metrics is limited.

Figure 19 clearly illustrates that performance metrics are equally optimized by both Combinatorial and ML resource management approaches, indicating the significance of these metrics. On the other hand, it is evident that ML-based approaches primarily focus on optimizing energy consumption metrics. This emphasis aligns with the fact that ML techniques are commonly used in migration and offloading approaches (as discussed in Q2). The primary objective of these strategies is to select resources during

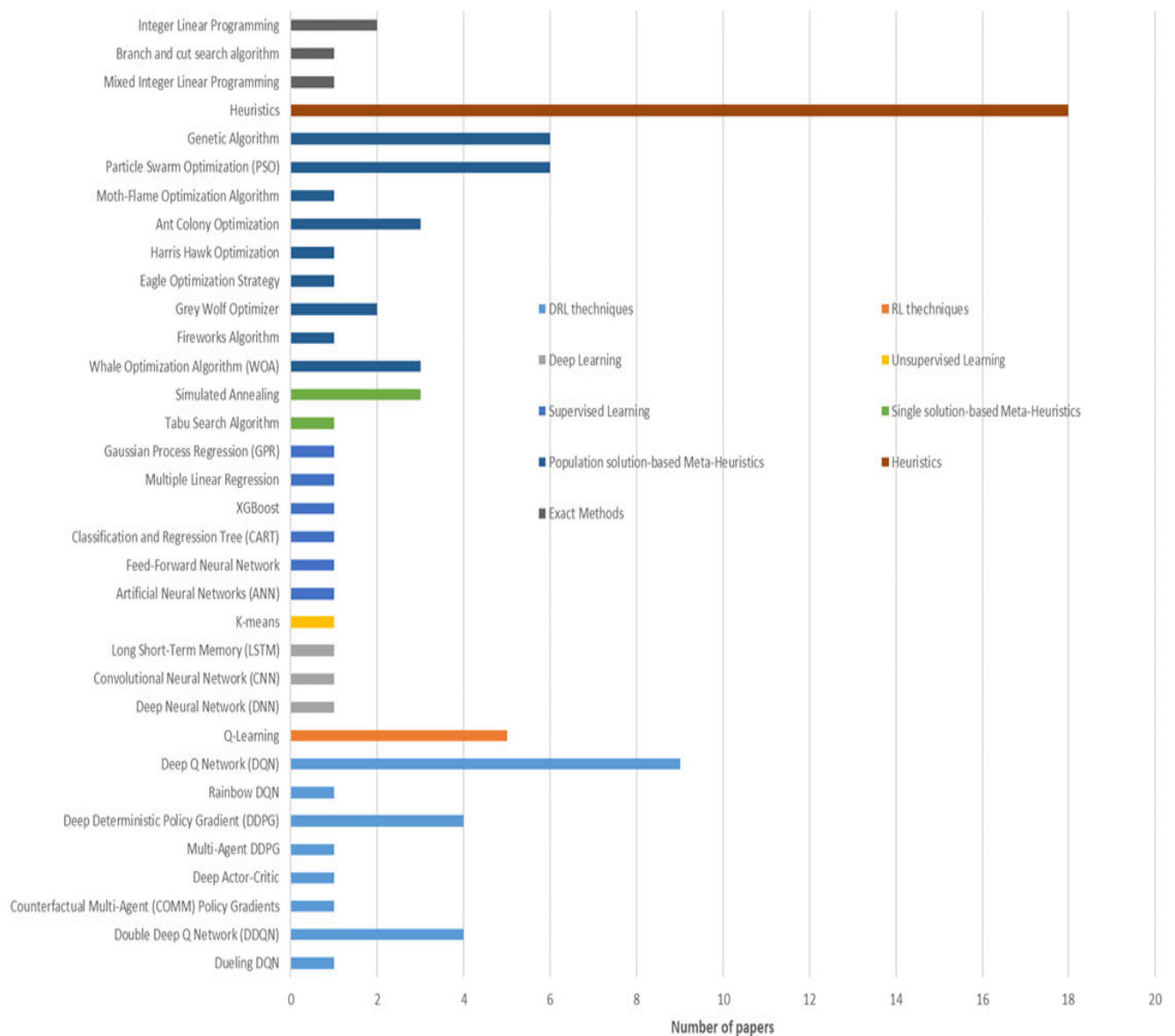


FIGURE 12. Classification based on resource management technique.

offloading or migration that are energy-efficient in terms of execution and transmission. Additionally, the Figure shows that cost metrics are mainly optimized by Combinatorial methods, which are primarily based on scheduling and resource allocation techniques. This reflects the focus of Combinatorial Optimization on optimizing resource usage and allocation in a cost-effective manner.

Q5: Do these approaches mostly consider a static or a dynamic environment? What are the different features considered in dynamic environments? Which approach, Combinatorial techniques, or ML, offers greater adaptability when dealing with dynamic changes in environment? Considering the changing environmental conditions over time, does this imply that resource allocation remains static or becomes dynamic?

In Figure 20, we can see plainly that 51% of the reviewed papers address a static environment while 49% consider a dynamic environment. We notice that related time-varying environment issues are challenging. Indeed, such environments are highly dynamic and the solution must be adapted on the fly accordingly. Figure 21 shows the different dynamic features invested in the studied works. We notice that user mobility is the feature the most addressed, with a percentage of 30%. Furthermore, we can see that resource availability time-varying is treated only by 8% of papers, which is a very low percentage since the availability metric is a key aspect of Fog/Edge resources. Moreover, we notice that other features are less considered such as network condition time-varying and dynamic task content.

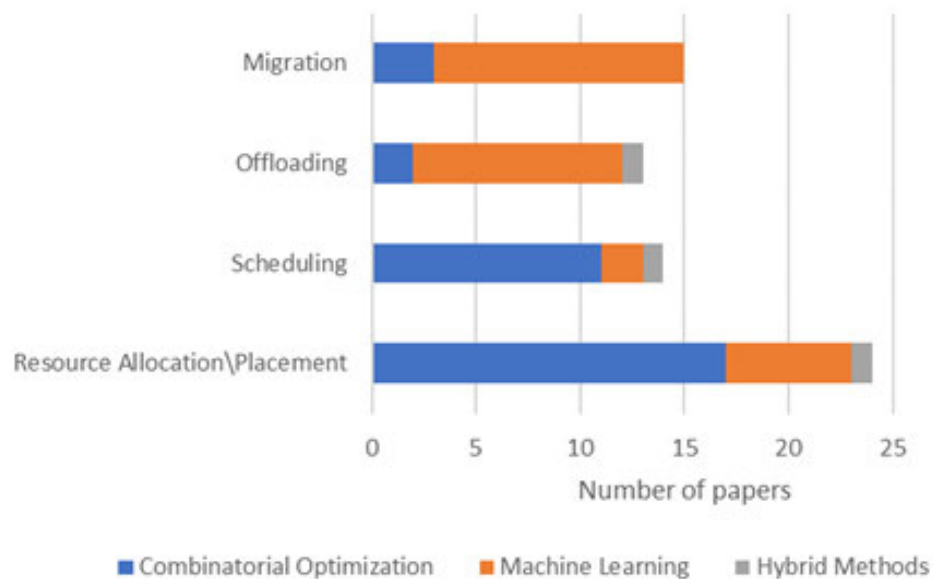


FIGURE 13. Classification based on resource management aspect.

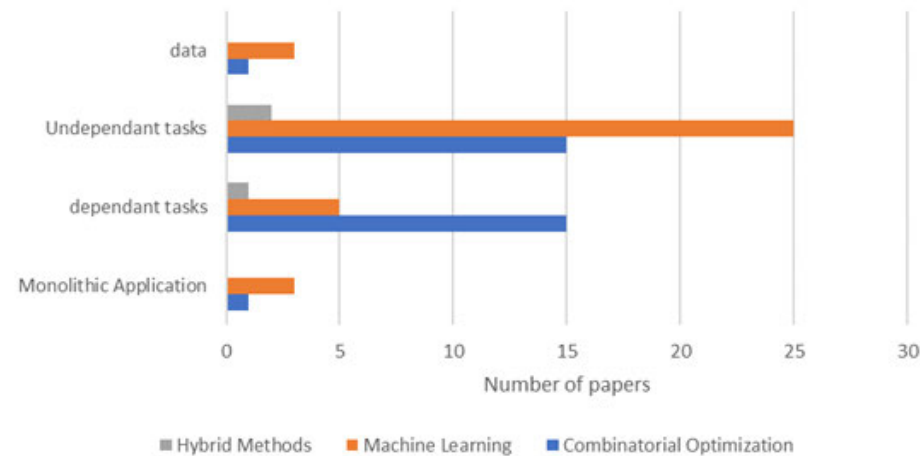


FIGURE 14. Classification based on resource management element.

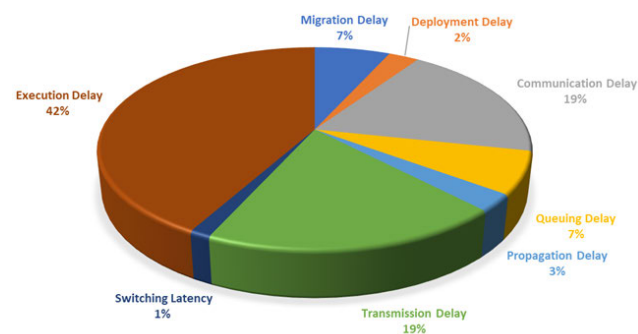


FIGURE 15. Comparison of performance-related metrics.

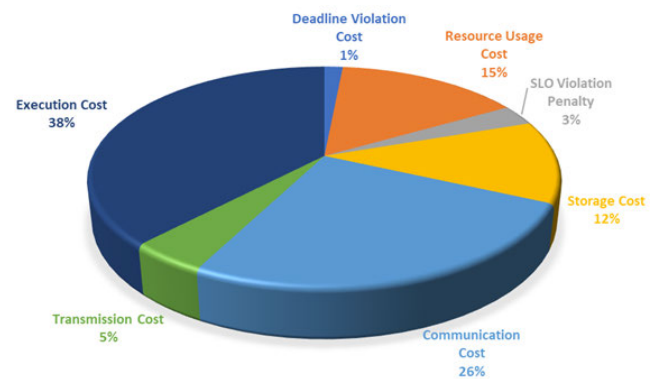


FIGURE 16. Comparison of cost metrics.

Figure 22 represents a classification of techniques according to environment nature. We can note that Combinatorial Optimization-based approaches highly address a static environment while the dynamic environment has been mostly

investigated using ML, more specifically Reinforcement and Deep Reinforcement Learning techniques. The main reason is that heuristic-based solutions cannot capture exhaustively

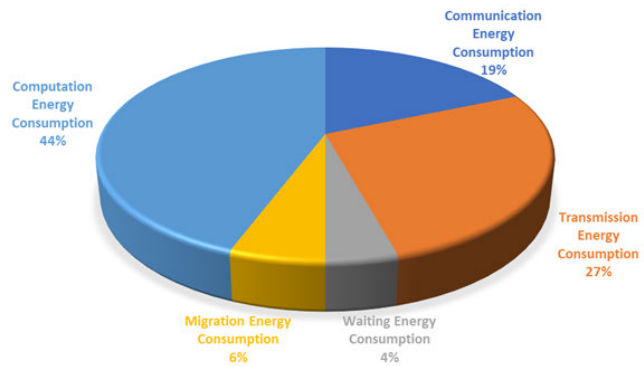


FIGURE 17. Comparison of energy metrics.

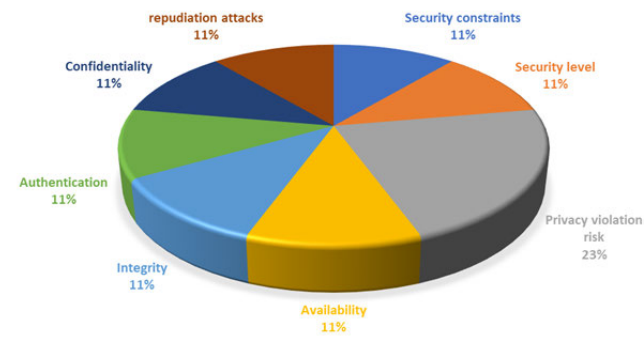


FIGURE 18. Comparison of security metrics.

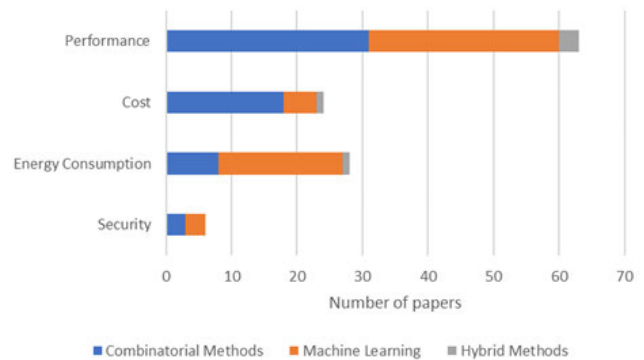


FIGURE 19. Technique-QoS classification.

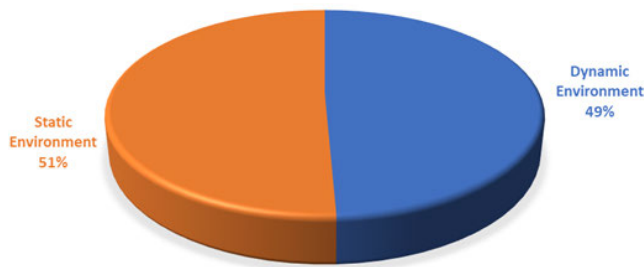


FIGURE 20. Static-dynamic environments.

all the changes and uncertainties that can occur in a dynamic environment. On the other hand, the RL and DRL can adjust the policy according to the environment changing and find the temporarily near-optimal decision for the long-term

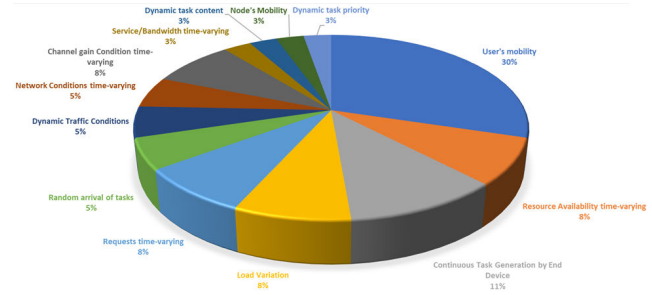


FIGURE 21. Dynamic environment-related features.

optimal solution. Moreover, they enable decision-making in a dynamic environment without any prior knowledge.

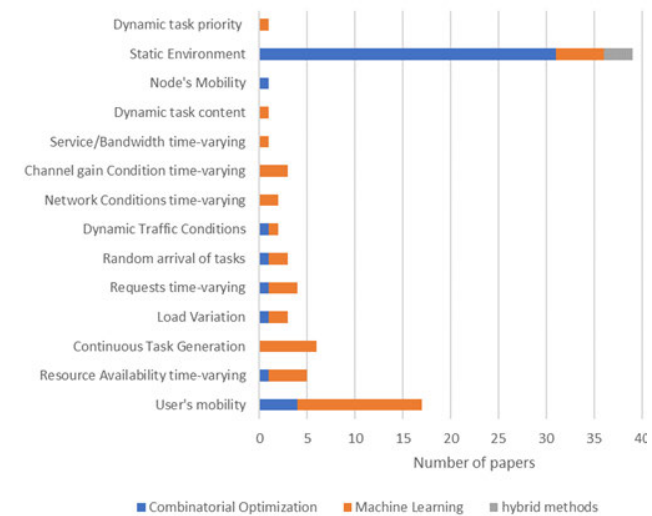


FIGURE 22. Technique-environment classification.

Figure 23 clearly demonstrates that in a dynamic environment, the allocation of resources remains largely static across Resource Allocation/Placement, Scheduling, and Offloading categories, with significantly higher values in the static environment columns compared to the dynamic environment ones. This challenges the assumption that dynamic environments inherently lead to dynamic resource allocation practices. Particularly striking is the contrast in the Offloading category, where static allocation dominates, suggesting a preference for predetermined resource allocation strategies even in dynamic contexts. Conversely, Figure 24 reinforces this observation by showcasing that static Allocation is prevalent across both static and dynamic environments, further highlighting the persistent reliance on static resource allocation approaches despite the dynamic nature of the environment. These findings underscore the need for a nuanced understanding of resource allocation strategies, recognizing that dynamic environments may not always translate into dynamic allocation practices, and that static allocation methods continue to hold significance even in dynamic scenarios depending on the context of scenarios and applications.

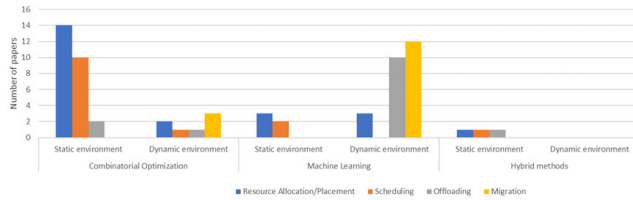


FIGURE 23. Aspects-environment type classification.

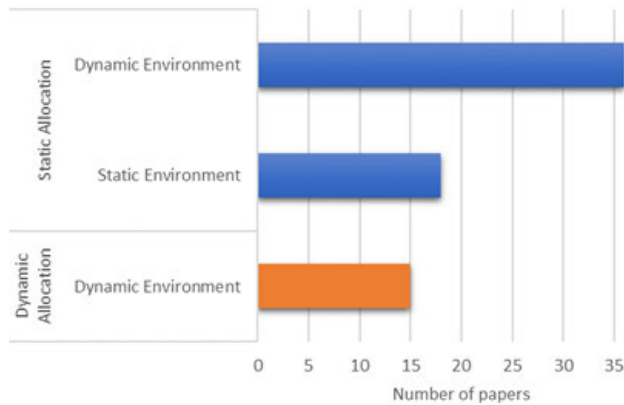


FIGURE 24. Allocation type-environment type classification.

Q6: What are the application domains of these approaches? Figure 25 illustrates the various application domains addressed in the literature. It shows clearly that IoT applications are the most adopted these last years. We also notice that mobile applications and healthcare are almost evenly considered. This is understandable due to the substantial roles of these domains in society's everyday lives.

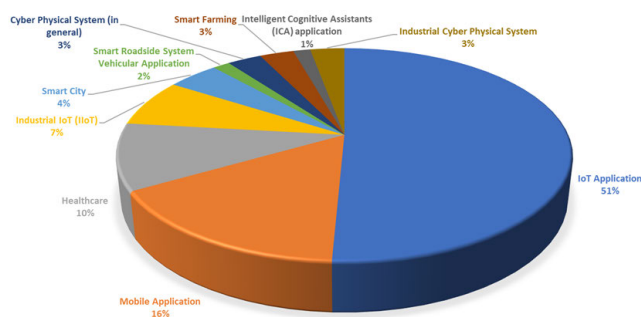


FIGURE 25. Application domains in resource management approaches.

Q7: What is the deployment environment the most considered in papers? During our study, we could note that Mobile Edge and Fog environments are almost evenly used for application deployment. As shown in Figure 26, they represent 17% and 17% of the total adopted deployment environments respectively. These paradigms hold attractive promise due to their close proximity to end-users, which enables them to enhance crucial QoS attributes, particularly in terms of reducing latency for time-sensitive applications. However, we can see plainly that the Fog-Cloud hybrid environments, constituting a significant 36% share, emerge as the most widely utilized deployment paradigm. This

dominance can be attributed to the inherent limitations of Fog environments, which face constraints in terms of resource capacities, making them unsuitable for executing highly computation-intensive tasks. To strike a balance between satisfying QoS requirements and meeting the computational demands (CPU, RAM) of tasks, the proposed solutions leverage the hybrid Fog-Cloud environment. This hybrid approach allows for maintaining an optimal trade-off between QoS satisfaction and task computation requirements.

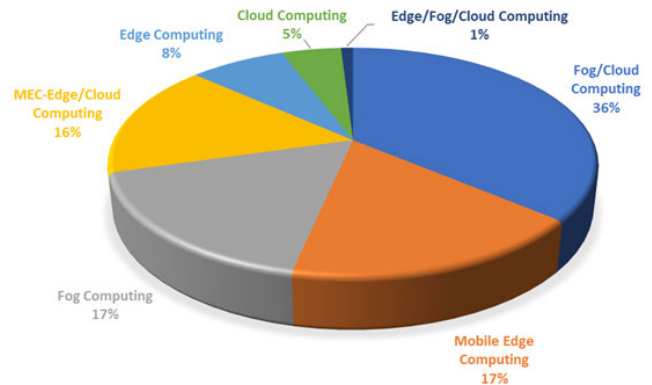


FIGURE 26. Consideration of deployment environments in the literature.

Q8: What computational tools are typically used to implement Combinatorial approaches compared to ML for resource management optimization? Figure 27 depicts the primary tools and programming languages commonly utilized in the literature. It is evident that the simulation tool IFogsim emerges as the most frequently employed software for Combinatorial-based approaches. IFogsim provides a robust platform for modeling and simulating real-time case studies, enabling the simulation of resource management and application scheduling policies across Edge and Cloud resources. This versatility allows it to accommodate diverse scenarios and conditions, making it a valuable tool for assessing designs against metrics like cost, network use, and perceived latency [120]. Additionally, Python stands out as a widely utilized programming language, particularly in ML-based approaches. This popularity stems from Python's extensive set of libraries that support numerous ML algorithms, making it a preferred choice for researchers and practitioners in the field.

Q9: Are experimental settings based on synthetic data or real data? How does the scalability of resources and tasks differ between Combinatorial Optimization and ML-based approaches in resource management? Is the consideration of static and dynamic environments a critical aspect in scalability? From Figure 28 we can see that 27 papers of combinatorial Optimization primarily use synthetic datasets, demonstrating the dependency on synthetic data for evaluating optimization methods. On the other hand, ML experiments show a more balanced approach, with a significant number using real datasets (10 instances), mixed (5 instances) alongside synthetic data. This suggests a recognition of the importance of real-world data in ML research. Conversely,

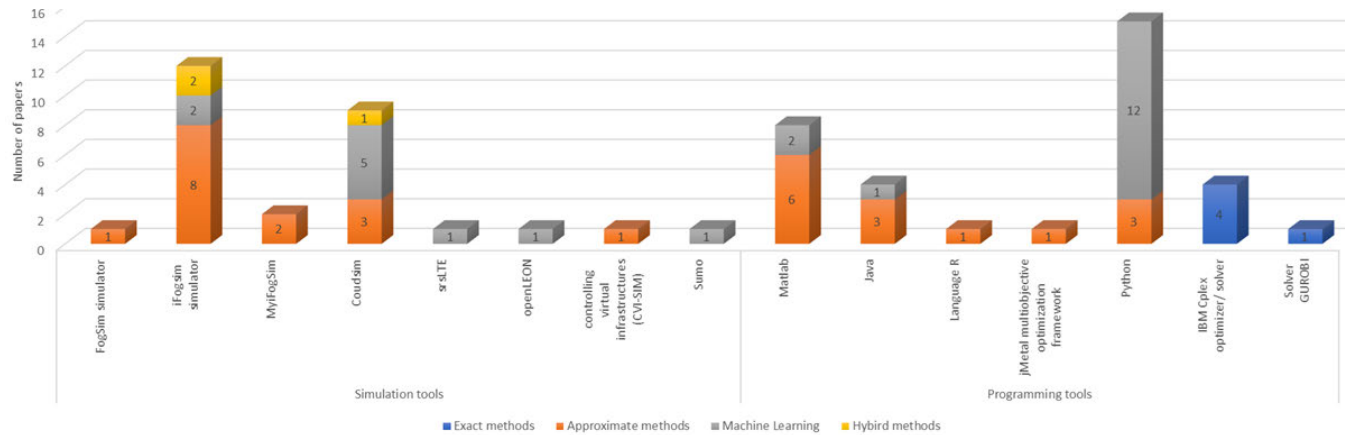


FIGURE 27. Comparison of software tools/languages.

Hybrid Methods predominantly utilize synthetic datasets (3 instances). These preferences are mainly influenced by the experimental context and the availability of real-world settings that can adequately mimic the study's context. The analysis of the number of resources in each study across different resource management approaches is demonstrated in Figure 29. In Combinatorial Optimization approaches, the number of resources varies widely, ranging from as low as 2 to as high as 800, with a median value around 20 resources. This suggests that Combinatorial Optimization approaches are applicable across a spectrum of resource-intensive scenarios. On the other hand, ML approaches show a wider distribution, with resource counts ranging from 1 to 500, indicating versatility in handling resource management tasks but with a notable concentration around 10 resources. Hybrid Methods demonstrate the highest resource counts, ranging from 60 to 513, which highlights the advantage of integrating multiple techniques. This integration allows these methods to handle more complex resource requirements.

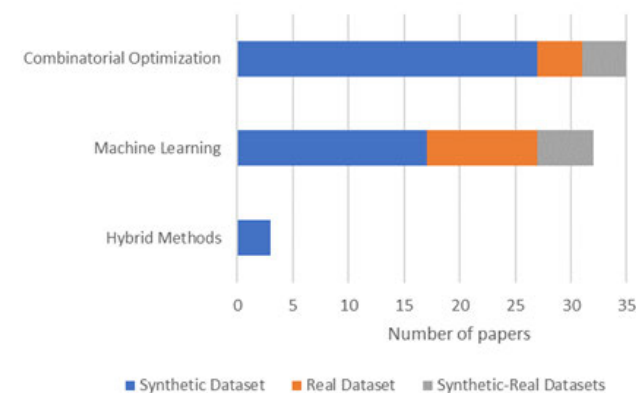


FIGURE 28. Synthetic/Real dataset classification.

We can clearly see from Figure 30 and Figure 31 that Combinatorial Optimization stands out for its scalability, managing applications with up to 2000 tasks per application

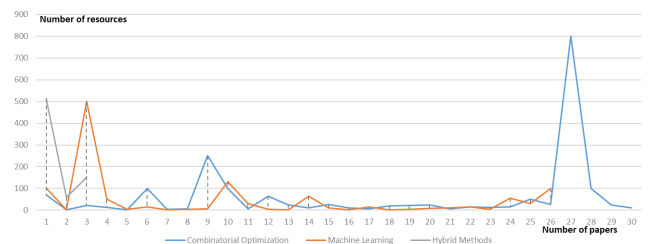


FIGURE 29. Resources scalability classification.

and accommodating up to 50000 IoT devices, demonstrating its effectiveness in handling large-scale deployments efficiently. On the other hand, ML-based approaches typically handle a moderate number of tasks varying from 9 to 5000. Hybrid methods also demonstrate effectiveness in managing a large number of tasks, with some instances handling up to 5000 tasks efficiently. In summary, combinatorial optimization methods excel in static environments, offering scalable resource and task management for large deployments. ML methods, adaptable to dynamic environments, provide flexible resource allocation but may struggle with resource-intensive cases due to scalability limits. Hybrid approaches, combining CO and ML techniques, handle a higher number of resources, integrating multiple strategies to address various resource management challenges effectively.



FIGURE 30. Application/tasks scalability classification.

Q10: What are the major limitations and challenges that exist in this field? Are there unique difficulties when integrating Combinatorial Optimization and ML techniques for optimizing resource management?

Efficiently allocating resources in Edge-Fog-Cloud Computing remains an ongoing hot area of investigation. Several

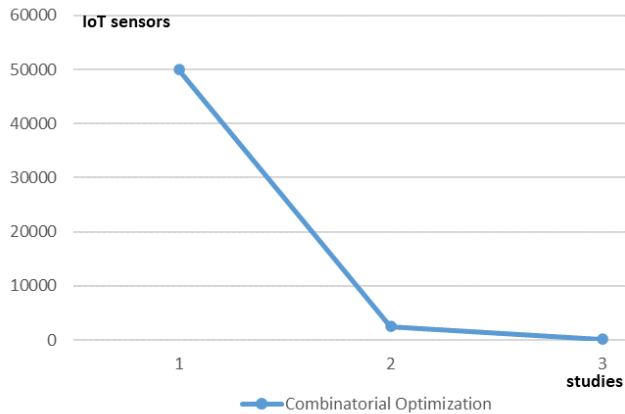


FIGURE 31. IoT data sensors scalability classification.

limitations and critical aspects are yet to be explored. Hereafter, we outline and discuss the limitations and challenges we have identified:

- **Technique management:** Exact methods strive to achieve optimal solutions, but their scalability is often hindered when tackling medium and large-scale problems [102]. The computational resources required render these methods less suitable for such scenarios. On the other hand, approximate-based methods do not encounter scalability issues. However, they grapple with the possibility of becoming trapped in local optimum, which means that a globally optimal solution is not guaranteed [44], [51]. In contrast, ML techniques, specifically RL and DRL, have demonstrated their effectiveness in predicting promising solutions [18], [42], [80]. It is important to consider that these techniques can be time and resource-intensive due to the substantial volume of data required for exploration and exploitation processes [83]. To address these limitations, there has been a growing interest in hybridization, which combines different approaches to leverage their strengths while mitigating their weaknesses. By integrating ML and approximate-based methods, a promising solution emerges for resolving resource management problems [121]. However, it should be noted that hybridization introduces a plethora of new parameters that define the hybrid scheme. Consequently, determining optimal parameter settings is a non-trivial task. A critical area for future investigation involves developing methods to automatically set these parameters in hybrid meta-heuristics [122]. Managing overfitting and ensuring generalization in ML models integrated with Combinatorial Optimization is crucial, necessitating strategies to prevent overfitting and ensure robust performance in real-world settings. Algorithm selection is another hurdle, as determining which Combinatorial and ML algorithms to integrate and how to effectively combine them is non-trivial, especially considering potential conflicts in optimization objectives [121], [123]. The hybridization process often leads to increased computational complexity,

impacting scalability and efficiency, particularly in large-scale resource management scenarios [124]. The reliance of ML techniques on large, high-quality datasets presents a challenge in resource-constrained environments, necessitating careful attention to data quality when integrating with Combinatorial Optimization [125]. Additionally, the integration may lead to less interpretable solutions, raising concerns about transparency and explainability, especially in critical applications.

- **Edge/Fog nodes dynamic content:** Edge/Fog nodes are known for their limited computing and storage capacities, making them constrained resources in Edge/Fog Computing. A significant limitation of current solutions [52], [60] is their focus on static properties, overlooking the dynamic nature and contextual variations of these nodes. This dynamic behavior introduces challenges in resource coordination and QoS management. Coordinating resources to consistently meet end-user QoS requirements is complex due to the unpredictable availability and mobility of Fog resources [126], further complicated by the heterogeneous behaviors of Edge and Fog nodes [29]. Many existing works assume static mobility or rely on prior knowledge of node behavior, which often does not align with real-life scenarios. In reality, unstable mobility patterns of Edge/Fog nodes can significantly degrade QoS for hosted services. Understanding the mobility behavior of heterogeneous Edge/Fog nodes is a challenging issue that requires dedicated research efforts [29].
- **User Mobility:** Most existing research has concentrated on achieving optimal or near-optimal solutions under the assumption of stationary users [87], [101], [102]. While these solutions perform well in static environments, their effectiveness declines in dynamic contexts involving continuous user movement, particularly in mobile applications. High-speed mobility, especially in vehicular scenarios, presents significant challenges for the successful deployment of MEC technology in IoT [127]. Mobile users traverse rapidly changing environments, necessitating a nuanced understanding of network condition variations and informed decision-making regarding near-future network states. This task is complex and requires meticulous consideration. Additionally, current mobility-based strategies often achieve satisfactory outcomes only when the mobile user's trajectory and the exact locations of Fog/Edge servers are known in advance [36], [37]. However, in real-world situations, this information is frequently uncertain or unavailable. Thus, there is a pressing need to adapt algorithms and techniques to manage these uncertainties effectively. One promising direction is the utilization of ML methods to predict mobile users' paths and speeds. By harnessing dynamic and realistic mobility patterns, ML can improve prediction accuracy, facilitating more effective optimization strategies in Fog/Edge systems.
- **Dependencies:** Currently, practical applications mainly rely on workflows, but existing research often overlooks

TABLE 6. Comparison of combinatorial optimization and machine learning approaches.

Work	Approach category	Algorithm type	Model-based algorithm	Tool/Language	Domain of Application	Experimental Evaluation		
						Data	MR	MA/MT
[101]	EM AM	H LP	ILP Heuristic LAMPH	Solver CPLEX iFogSim Java	IoT	S	70	MA/MT 100
[102]	EM	LP	ILP	IBM Cplex optimization	Smart Farming	S	2	MA 7/MT 3
[87]	EM	LP	MILP	Solver GUROBI	IoT	S	21	MT 50
[103]	AM	H	MM Heuristic	FogSim	IoT	S	13	MA/MT 6
[68]	AM	LP MH	ILP Greedy heuristic PSO	IBM CPLEX Optimizer	Smart Roadside System	S	3	MA 2
[104]	AM	H	Risk minimizing Heuristic Cost minimizing Heuristic	Java	Healthcare	S	100	MBP 5MT 200
[105]	AM	H	LAMP Heuristic	iFogSim	Healthcare	S	5	MA 8/MT 3
[106]	AM	H	MIDAF Heuristic	iFogSim	IoT	S	6	MA 5/MS 4
[88]	AM	H	Virtual machine migration scheme Heuristic	MyiFogSim	IoT	S	250	MA 21
[38]	AM	H	Greedy Heuristic	language R	Industrial IoT (IIoT)	S	MO 100	MA/MT 50
[44]	AM	H	MG Heuristic	MyiFogSim	IoT	S-R	7	MA 100/MS 3
[70]	AM	MH	Min-CCV Heuristic	Matlab	IoT	S	65	MA 1/MT 300
[71]	AM	H	Min-V Heuristic PSG Heuristic PSG-M Heuristic TSOO Heuristic	C++	IoT	S	25	MT 1000
[53]	AM	H	PTS-Ra Heuristic	iFogSim	Healthcare	R	11	MIoT 5000
[90]	AM	H	DCSP Heuristic	iFogSim	IoT	S	2	MT 200
[96]	AM	H	IFSO	Java	IoT	S	26	MS 50
[43]	AM	MH	GA	Matlab	IoT	S	10	MA 5/MS 5
[51]	AM	MH	MAPO-based genetic algorithm	Python	Smart City	R	16	MT 8152
[107]	AM	MH	MFO	JMetal multi-objective optimization framework	Healthcare	R	12	A 3/MT 30
[60]	AM	MH	ACO	iFogSim toolkit	CPS	S	30	MA 1/MT 1000
[52]	AM	MH	PSO	Matlab	Smart City	S	21	MIoT 2000
[72]	AM	MH	CAG	iFogSim	IIoT	S	25	MT 1000
[73]	AM	MH	Fog-AMOSM	CloudSim 5.0	CPS	S	6	MT 160
[74]	AM	MH	PSO	MATLAB	IoT	S	16	MS 613
[108]	AM	MH	DSRP	iFogSim	IoT	S	13	MA 1/MT 5
[75]	AM	MH	PSO	CloudSim 3	IoT	R	16	MT 8152
[7]	AM	MH	TS	controlling virtual infrastructures (CVI-SIM)	IoT	S	50	MA 10/MT 5
[69]	AM	MH	Branch-and-Cut search algorithm	IBM CPLEX 12.8	IoT	S	26	MIoT 2500
[112]	AM	MH	SA	MATLAB	IoT	S	26	MIoT 2500
[113]	AM	MH	Fireworks algorithm	Java	IoT	S-R	800	MW 4/MT 800
[97]	AM	MH	ACO	CloudSim 5.0	IoT	S	100	MW 10/MT 500
[114]	AM	MH	multi-objective cost-aware discrete grey wolf optimization (MoDGWA)	MATLAB	IoT	S	25	MT 200
[99]	AM	MH	Enhanced Harris Hawk Optimization	WorkflowSim	IoT	S/R	4	WT/MT 1000
[109]	AM	HMH	whale optimization algorithm grey wolf optimization (GWO)	Python	IoT	S	22	500
[85]	AM	HMH	ACO SA GA	JAVA	IoT	S	10	MS 9
[16]	DB	SL	Gaussian Process Regression for Fog-Cloud Allocation algorithm	Python3.6	Mobile application	R/S	/	/
[76]	DB	USL	K-Means	iFogSim Python	Mobile application	S	MT 102	MA 2/MT 6
[40]	DB	USL	Classification and regression tree Algorithm	iFogSim CloudSim	Smart Farming	/	3	/
[77]	DB	SL	ANN	CloudSim	IoT	R	500	MT 500
[78]	DB	SL	Multiple Linear Regression	MATLAB	Healthcare	S	/	MT 1000
[17]	DB	DL	Improved LSTM	CloudSim	IoT	S-R	50	MA 50/MT 10
[8]	DB	DL	DNN	Python	IIoT	R	3	MS 380
[91]	DB	DL	DNN	Java Python CloudSim	Healthcare	R	4	MA 2/MT 800
[41]	ARB	RL	CNN	CloudSim	Mobile application	R	16	MA 1/MT 1024
[45]	ARB	RL	Q-learning	IoT	IoT	S	3	MT 9
[79]	ARB	RL	RL based task scheduling with the clustering	SUMO MATLAB	IoT	S	5	MT 5000
[89]	ARB	RL	Q-learning	CloudSim	IoT	R	6	MS 8
[100]	ARB	RL	Q-learning	CloudSim srlLTE	IoT	S-R	130	MT 2000
[80]	ARB	DRL	DQN	openLEON	IoT	S-R	12	MT 100
[110]	ARB	DRL	MB-DDPG	Python 3.6	Vehicle application	S	30	/
[86]	ARB	DRL	DQN	Python (Using Keras)	IoT	R-S	4	MA 20/MS 15
[81]	ARB	DRL	DDPG	Python (Using TensorFlow)	Mobile application	R	MA 2/MT 3	/
[82]	ARB	DRL	Deep Recurrent Actor-Critic	Python (using Tensor Flow)	Healthcare	S	3	MT 20
[6]	ARB	DRL	DQN	using TensorFlow	Mobile application	R	64	MT 18
[92]	ARB	DRL	DQN		Mobile application	S	10	MS 30
[18]	ARB	DRL	COMA		Mobile application	S	1	MS 6
[83]	ARB	DRL	DDPG		Mobile application	S	16	MT 80
[36]	ARB	DRL	DQN		IIoT	S	1	MT 400
[61]	ARB	DRL	DDQN		Mobile Application	S	4	MA 4
[37]	ARB	DRL	DQN		Mobile Application	R	8	MT 800
[49]	ARB	DRL	DDPG		IoT	R	10	MA 40
[93]	ARB	DRL	DDQN		IoT	S	16	MT 10
[94]	ARB	DRL	Rainbow DQN		IoT	S	5	MT 555
[98]	ARB	DRL	MADDPG	Python (PyTorch)	Vehicle	S	5	MT 555
[115]	ARB	DRL	DQN		Vehicle	S	MA 30	/
[42]	HML	DRL	DQN		IoT	S	6	MT 150
[84]	HML	DRL	DDQN		IoT	R	6	MT 1000
[95]	HML	DRL	KNN		IoT	R	55	MT 500
[39]	HM	H	Profit based Assignment heuristic	TensorFlow	Mobile Application	S	31	MA 30
[111]	HM	SL	Q-learning XGBOOST	Python	IoT	S	100	MA 100
[47]	HM	SL	WOA	iFogSim	Intelligent Cognitive Assistants (ICA) application	S	513	MT 500
		SL	Eagle optimization	Testbed iFogSim	IoT	S	60	MA 2/MS 10
		MH	ANN BPSO	CloudSim-3.0.3	IoT	S	150	MT 5000

task dependencies, focusing instead on independent tasks [73], [79], [103]. While resource allocation and scheduling methods have addressed dependent tasks [102], [103], task graph offloading or migration has received limited attention due to the complexities of task dependencies and data transfer requirements [48]. Dealing with task dependencies in offloading presents challenges such as deciding whether a task should be executed locally or remotely, considering the limited energy budget of IoT devices. Additionally, it is crucial to assess the impact of current offloading decisions on subsequent

tasks. In continuous migration scenarios, determining the workflow parts affected by each migration step is challenging. Understanding the implications of current migration decisions on the overall objective is also essential. Therefore, further research is needed to address these issues and develop robust solutions.

- **Security and Privacy:** Current resource management research mainly focuses on optimal solutions for deadline-sensitive applications in Fog and Edge Computing [44], [71]. Future research should prioritize security-sensitive applications used by enterprises and organizations.

TABLE 7. Classification based on the optimized metrics.

Works	Global	Performance										EC				Cost					Energy Consumption					Security
		ET	CT	MT	TT	ST	QT	DT	PT	SWT	MTC	CPUC	RAMC	SC	CC	TC	RU	DVC	SLOVP	EEC	CEC	TEC	MEC	WEC		
[101]	Latency																									
[102]	Delay	✓	✓								✓			✓	✓									✓		
[87]	Timespan	✓										✓	✓		✓											
[103]	Latency																✓									
[68]	Delay	✓	✓					✓									✓									
[104]											✓			✓	✓									✓		
[105]	Latency	✓	✓																							
[06]	Response time	✓	✓					✓			✓															
[88]	Latency	✓																								
[38]	Latency			✓											✓									✓		
[44]	Delay violations																			✓	✓					
[70]											✓			✓	✓			✓								
[71]	Response time	✓			✓			✓		✓										✓						
	Deadline violation time																			✓						
[53]															✓		✓									
[90]	Latency	✓			✓			✓																		
[96]	Delay	✓	✓						✓																	
[43]	Makespan	✓	✓								✓															
[51]	Latency	✓	✓																							
[107]	Completion time	✓	✓											✓	✓					✓	✓					
[60]	Makespan	✓	✓																							
[52]	Response time	✓	✓																							
[72]																										
[73]	Completion time	✓										✓			✓											
[74]	Response time	✓										✓				✓			✓							
[08]	Response time	✓																								
[75]	Makespan	✓							✓																	
[7]	Delay	✓	✓					✓		✓								✓								
[69]	Latency	✓	✓	✓																						
[112]	Makespan	✓	✓																	✓	✓					
[113]	Latency																			✓						
[97]	Makespan	✓										✓	✓		✓											
[14]	Makespan	✓																✓								
[109]	Latency		✓																							
[85]	Latency																			✓	✓					
[99]	Latency	✓			✓															✓		✓				
[16]	Latency																			✓						
[76]	delay																			✓						
[40]											✓	✓	✓				✓							✓		
[77]	Response time																									
[78]	Delay	✓																		✓	✓					
[17]																		✓			✓					
[8]	Response time																			✓		✓				
[9]	Delay	✓			✓															✓		✓				
[41]	Latency	✓			✓															✓		✓				
[79]	Delay																	✓				✓				
[89]	Response time																			✓						
[100]	Delay	✓			✓			✓														✓		✓		
[45]	Makespan														✓							✓				
[80]																										
[110]	Response time	✓		✓	✓																					
[86]	Completion Time											✓	✓	✓												
[81]	Latency	✓			✓															✓		✓				
[82]	Delay	✓		✓	✓																					
[6]	Response time	✓			✓																					
[92]	Latency				✓		✓			✓																
[18]	Average completion time	✓		✓	✓																					
[83]	Delay	✓		✓	✓																	✓	✓			
[36]	Delay	✓		✓	✓																	✓	✓			
[61]	Latency	✓			✓															✓		✓				
[37]	Delay	✓		✓	✓																	✓	✓			
[49]	Delay	✓	✓		✓															✓	✓	✓	✓			
[93]	Delay	✓	✓																	✓	✓					
[94]	Delay	✓	✓																	✓	✓					
[98]	Delay	✓	✓		✓																✓					
[115]	Makespan	✓	✓																					✓		
[42]	Latency	✓			✓			✓												✓		✓				
[84]	Delay	✓			✓															✓				✓		
[95]	Delay	✓		✓	✓																✓					
[39]	Delay	✓			✓																					
[111]	Response time		✓					✓				✓								✓		✓				
[47]	Compilation time																									

Although Edge and Fog Computing offer advantages over traditional Cloud systems, certain security issues hinder widespread adoption [3]. Edge/Fog resources are vulnerable to attacks like Denial of Service (DoS), which disrupt service access for legitimate users. Protecting individual and enterprise privacy is also challenging, especially with Fog nodes near users. Insufficient security during data transmission can lead to unauthorized access and data theft [128]. Blockchain techniques have increasingly been adopted to enhance security in Fog computing due to their capabilities in ensuring data integrity, decentralization, and traceability. Recent studies have explored the use of Blockchain and smart contracts in multi-user/server MEC-enabled 5G and wireless networks to secure edge-to-cloud real-time task offloading [100]. Cooperative Blockchain-based frameworks have also been proposed to address authentication and privacy concerns in distributed IoT environments [112]. However, despite their promise, practical integration of Blockchain

remains an evolving area, facing challenges such as slow and cumbersome execution, scalability bottlenecks, high development costs, and increased energy consumption [129], [130]. Conversely, ML techniques show promise in predicting IoT attacks [131], [132]. Thus, further research should focus on predicting Fog node security levels and potential attacks to inform application or data allocation.

- **Energy management:** Energy management poses a critical challenge, particularly in migration and offloading approaches. Lightweight IoT devices and mobile Fog nodes face the significant issue of energy consumption due to frequent storage, computation, and task transmission processes. This challenge is particularly daunting for battery-powered devices, as the current advancements in battery technology are unable to meet the energy demands of IoT applications [85]. Consequently, without alternative energy supply methods or improved energy efficiency, the transmission of data and applications in such scenarios will be severely limited. To effectively tackle these challenges,

TABLE 8. Classification based on the nature of environment.

Work	Static Environment	Dynamic Environment												
		EUf	NF	NF	AF	AF	NWF	NWF	EUf	NF	NF	AF	AF	AF
		User's Mobility	Node's Mobility	Resource Availability time-varying	Continuous Task Generation by End Device	Dynamic arrival of tasks	Channel In Condition time-varying	Network Conditions time-varying	Requests time-varying	Load time-variation	Dynamic Traffic Conditions	Service' Bandwidth time-varying	Dynamic task content	Dynamic task priority
[101]	✓													
[102]	✓													
[87]	✓													
[103]	✓													
[68]	✓	✓	✓	✓						✓				
[104]	✓													
[105]	✓													
[106]	✓													
[88]	✓	✓												
[38]	✓													
[44]	✓	✓												
[70]	✓													
[71]	✓													
[53]	✓					✓								
[90]	✓													
[96]	✓													
[43]	✓													
[51]	✓													
[107]	✓													
[60]	✓													
[52]	✓													
[72]	✓													
[73]	✓													
[74]	✓													
[108]	✓													
[75]	✓										✓			
[7]	✓													
[69]	✓								✓					
[112]	✓													
[113]	✓													
[97]	✓													
[114]	✓													
[109]	✓	✓												
[85]	✓													
[99]	✓													
[116]	✓													
[76]	✓													
[40]	✓					✓								
[77]	✓													
[78]				✓										
[17]												✓		
[8]								✓			✓			
[91]					✓					✓				
[41]	✓						✓							
[45]	✓													
[79]	✓								✓					
[89]	✓													
[100]		✓			✓									
[80]		✓												
[110]		✓		✓					✓					
[86]		✓		✓				✓						
[81]		✓												
[82]		✓												
[6]									✓					
[92]						✓								
[18]		✓												
[83]					✓									
[36]		✓												
[61]		✓												
[37]		✓												
[49]		✓												
[93]		✓												
[94]		✓			✓									
[98]							✓							
[115]														✓
[42]					✓					✓				
[84]		✓		✓	✓		✓							
[95]				✓										
[39]	✓													
[111]	✓													
[47]	✓													

further research must be dedicated to devising resource management strategies capable of mitigating the impact of energy constraints on IoT devices and mobile Fog nodes.

VII. CONCLUSION

This paper presented a comprehensive analysis of recent advancements in the field of resource management, specifically focusing on resource allocation, scheduling, offloading, and migration within the context of Edge-Fog-Cloud Computing. This study encompasses the use of Combinatorial

Optimization and ML techniques. Based on the taxonomy we have proposed, first, the approaches are reviewed based on a set of dimensions including the nature of the considered elements concerned by resource management approaches, the handled aspects, and the addressed deployment environment. In addition, a classification according to the different used techniques, specifically related to Combinatorial and ML, is given. Thereafter, the handled QoS parameters and the considered environment type are studied and discussed. Based on the provided study, we highlighted the limitations

and the major challenges of the existing solutions that required further exploration and investigation.

In our future work, we plan to extend this study by addressing the concept of Quality of Experience (QoE) awareness for end-users in resource allocation and application scheduling. The aim is to study user-centric approaches that prioritize not only resource management but also the overall experience and satisfaction of end users. We also plan to extend our previous work [50] by considering the edge device battery limitations and developing a partial offloading and necessary migration strategy that adapts to energy constraints while maintaining QoS requirements.

REFERENCES

- [1] R. Mahmud, K. Ramamohanarao, and R. Buyya, "Application management in fog computing environments: A taxonomy, review and future directions," *ACM Comput. Surv.*, vol. 53, no. 4, pp. 1–43, Jul. 2021.
- [2] M. Songhorabadi, M. Rahimi, A. MoghadamFarid, and M. Haghi Kashani, "Fog computing approaches in IoT-enabled smart cities," *J. Netw. Comput. Appl.*, vol. 211, Feb. 2023, Art. no. 103557.
- [3] A. M. Alwakeel, "An overview of fog computing and edge computing security and privacy issues," *Sensors*, vol. 21, no. 24, p. 8226, Dec. 2021.
- [4] N. R. Tadapaneni, "Role of fog computing in the Internet of Things," *Int. J. Sci. Res. Eng. Trends*, vol. 5, no. 6, pp. 2279–2281, Jan. 2019.
- [5] A. C. Caminero and R. Muñoz-Mansilla, "Quality of service provision in fog computing: Network-aware scheduling of containers," *Sensors*, vol. 21, no. 12, p. 3978, Jun. 2021.
- [6] Y. Hao, M. Chen, H. Gharavi, Y. Zhang, and K. Hwang, "Deep reinforcement learning for edge service placement in software-defined industrial cyber-physical system," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5552–5561, Aug. 2021.
- [7] A. Leivadreas, G. Kesidis, M. Ibnkahla, and I. Lambadaris, "VNF placement optimization at the edge and cloud," *Future Internet*, vol. 11, no. 3, p. 69, Mar. 2019.
- [8] A. Lakhan, Q.-U.-A. Mastoi, M. Elhoseny, M. S. Memon, and M. A. Mohammed, "Deep neural network-based application partitioning and scheduling for hospitals and medical enterprises using IoT assisted mobile fog cloud," *Enterprise Inf. Syst.*, vol. 16, no. 7, pp. 1–23, Jul. 2022.
- [9] R. O. Aburukba, T. Landolsi, and D. Omer, "A heuristic scheduling approach for fog-cloud computing environment with stationary IoT devices," *J. Netw. Comput. Appl.*, vol. 180, Apr. 2021, Art. no. 102994.
- [10] M. Abdel-Basset, D. El-Shahat, M. Elhoseny, and H. Song, "Energy-aware Metaheuristic algorithm for industrial-Internet-of-Things task scheduling problems in fog computing applications," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12638–12649, Aug. 2021.
- [11] F. Peres and M. Castelli, "Combinatorial optimization problems and metaheuristics: Review, challenges, design, and development," *Appl. Sci.*, vol. 11, no. 14, p. 6449, Jul. 2021.
- [12] S. Voß, S. Martello, I. H. Osman, and C. Roucairol, *Meta-heuristics: Advances and Trends in Local Search Paradigms for Optimization*. Springer, 2012.
- [13] M. W. Berry, A. Mohamed, and B. W. Yap, *Supervised and Unsupervised Learning for Data Science*. Cham, Switzerland: Springer, 2019.
- [14] P. P. Shinde and S. Shah, "A review of machine learning and deep learning applications," in *Proc. 4th Int. Conf. Comput. Commun. Control Autom. (ICCUBE)*, Aug. 2018, pp. 1–6.
- [15] M. Mehdi Afsar, T. Crump, and B. Far, "Reinforcement learning based recommender systems: A survey," 2021, *arXiv:2101.06286*.
- [16] R. A. C. da Silva and N. L. S. D. Fonseca, "Resource allocation mechanism for a fog-cloud infrastructure," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2018, pp. 1–6.
- [17] Y. Wang, X. Wan, X. Du, X. Chen, and Z. Lu, "A resource allocation strategy for edge services based on intelligent prediction," in *Proc. IEEE 6th Int. Conf. Smart Cloud (SmartCloud)*, Nov. 2021, pp. 78–83.
- [18] C. Liu, F. Tang, Y. Hu, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, Jul. 2021.
- [19] J. Zhang and D. Tao, "Empowering things with intelligence: A survey of the progress, challenges, and opportunities in artificial intelligence of things," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 7789–7817, May 2021.
- [20] K. H. Abdulkareem, M. A. Mohammed, S. S. Gunasekaran, M. N. Al-Mhiqani, A. A. Mutlag, S. A. Mostafa, N. S. Ali, and D. A. Ibrahim, "A review of fog computing and machine learning: Concepts, applications, challenges, and open issues," *IEEE Access*, vol. 7, pp. 153123–153140, 2019.
- [21] S. S. Gill et al., "AI for next generation computing: Emerging trends and future directions," *Internet Things*, vol. 19, Aug. 2022, Art. no. 100514.
- [22] Z. Chang, S. Liu, X. Xiong, Z. Cai, and G. Tu, "A survey of recent advances in edge-computing-powered artificial intelligence of things," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 13849–13875, Sep. 2021.
- [23] S. Tuli, F. Mirhakimi, S. Pallevatta, S. Zawad, G. Casale, B. Javadi, F. Yan, R. Buyya, and N. R. Jennings, "AI augmented edge and fog computing: Trends and challenges," *J. Netw. Comput. Appl.*, vol. 216, Jul. 2023, Art. no. 103648.
- [24] M. Ghobaei-Arani, A. Souri, and A. A. Rahmani, "Resource management approaches in fog computing: A comprehensive review," *J. Grid Comput.*, vol. 18, no. 1, pp. 1–42, Mar. 2020.
- [25] M. S. Raghavendra, P. Chawla, and A. Rana, "A survey of optimization algorithms for fog computing service placement," in *Proc. 8th Int. Conf. Rel., INFOCOM Technol. Optim. Trends Future Directions (ICRITO)*, Jun. 2020, pp. 259–262.
- [26] A. Hedhli and H. Mezni, "A survey of service placement in cloud environments," *J. Grid Comput.*, vol. 19, no. 3, pp. 1–32, Jun. 2021.
- [27] I. B. Lahmar and K. Boukadi, "Resource allocation in fog computing: A systematic mapping study," in *Proc. 5th Int. Conf. Fog Mobile Edge Comput. (FMEC)*, Apr. 2020, pp. 86–93.
- [28] R. Belmahdi, D. Mechta, and S. Harous, "A survey on various methods and algorithms of scheduling in fog computing," *Ingénierie Des Syst. D'Inf.*, vol. 26, no. 2, pp. 211–224, Apr. 2021.
- [29] N. E. H. Boubaker, K. Zarour, N. Guermouche, and D. Benmerzoug, "Fog and edge service migration approaches based on machine learning techniques: A short survey," in *Proc. TACC*, 2022, pp. 33–44.
- [30] S. M. A. Huda and S. Moh, "Survey on computation offloading in UAV-enabled mobile edge computing," *J. Netw. Comput. Appl.*, vol. 201, May 2022, Art. no. 103341.
- [31] K. Ostrowski, K. Matecki, P. Dziurzański, and A. K. Singh, "Mobility-aware fog computing in dynamic networks with mobile nodes: A survey," *J. Netw. Comput. Appl.*, vol. 219, Aug. 2023, Art. no. 103724.
- [32] M. Hosseinzadeh, E. Azhir, J. Lansky, S. Mildeeva, O. H. Ahmed, M. H. Malik, and F. Khan, "Task scheduling mechanisms for fog computing: A systematic survey," *IEEE Access*, vol. 11, pp. 50994–51017, 2023.
- [33] G. K. Walia, M. Kumar, and S. S. Gill, "AI-empowered fog/edge resource management for IoT applications: A comprehensive review, research challenges, and future perspectives," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 619–669, 1st Quart., 2024.
- [34] A. Choudhury, M. Ghose, A. Islam, and Yogita, "Machine learning-based computation offloading in multi-access edge computing: A survey," *J. Syst. Archit.*, vol. 148, Mar. 2024, Art. no. 103090.
- [35] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," *Softw. Eng. Group, Keele Univ., Durham Univ. Joint, Durham, U.K., Tech. Rep. EBSE-2007-01*, 2007.
- [36] W. Chen, Y. Chen, J. Wu, and Z. Tang, "A multi-user service migration scheme based on deep reinforcement learning and SDN in mobile edge computing," *Phys. Commun.*, vol. 47, Aug. 2021, Art. no. 101397.
- [37] S. Lu, J. Wu, J. Shi, P. Lu, J. Fang, and H. Liu, "A dynamic service placement based on deep reinforcement learning in mobile edge computing," *Network*, vol. 2, no. 1, pp. 106–122, Feb. 2022.
- [38] V. Casola, A. De Benedictis, S. Di Martino, N. Mazzocca, and L. L. L. Starace, "Security-aware deployment optimization of cloud-edge systems in industrial IoT," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12724–12733, Aug. 2021.
- [39] T. Sen and H. Shen, "Machine learning based timeliness-guaranteed and energy-efficient task assignment in edge computing systems," in *Proc. IEEE 3rd Int. Conf. Fog Edge Comput. (ICFEC)*, May 2019, pp. 1–10.
- [40] D. Rahbari and M. Nickray, "Task offloading in mobile fog computing by classification and regression tree," *Peer-Peer Netw. Appl.*, vol. 13, no. 1, pp. 104–122, Jan. 2020.
- [41] X. Liu, Z. Qin, and Y. Gao, "Resource allocation for edge computing in IoT networks via reinforcement learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–6.

- [42] Y. Tu, H. Chen, L. Yan, and X. Zhou, "Task offloading based on LSTM prediction and deep reinforcement learning for efficient edge computing in IoT," *Future Internet*, vol. 14, no. 2, p. 30, Jan. 2022.
- [43] R. Xu, Y. Wang, Y. Cheng, Y. Zhu, Y. Xie, A. S. Sani, and D. Yuan, "Improved particle swarm optimization based workflow scheduling in cloud-fog environment," in *Proc. Int. Conf. Bus. Process Manage.*, Jan. 2019, pp. 337–347.
- [44] T. Djemai, P. Stolf, T. Monteil, and J. Pierson, "Mobility support for energy and QoS aware IoT services placement in the fog," in *Proc. Int. Conf. Softw.*, Sep. 2020, pp. 1–7.
- [45] M. K. Pandit, R. N. Mir, and M. A. Chishti, "Adaptive task scheduling in IoT using reinforcement learning," *Int. J. Intell. Comput. Cybern.*, vol. 13, no. 3, pp. 261–282, Jun. 2020.
- [46] S. Yassa, R. Chelouah, H. Kadima, and B. Granado, "Multi-objective approach for energy-aware workflow scheduling in cloud computing environments," *Sci. World J.*, vol. 2013, no. 1, 2013, Art. no. 350934.
- [47] M. I. Alghamdi, "Optimization of load balancing and task scheduling in cloud computing environments using artificial neural networks-based binary particle swarm optimization (BPSO)," *Sustainability*, vol. 14, no. 19, p. 11982, Sep. 2022.
- [48] B. Xu, Y. Hu, M. Hu, F. Liu, K. Peng, and L. Liu, "Iterative dynamic critical path scheduling: An efficient technique for offloading task graphs in mobile edge computing," *Appl. Sci.*, vol. 12, no. 6, p. 3189, Mar. 2022.
- [49] N. E. H. Boubaker, K. Zarour, N. Guermouche, and D. Benmerzoug, "Double deep Q-network-based time and energy-efficient mobility-aware workflow migration approach," in *Proc. Int. Conf. Cooperat. Inf. Syst.*, Oct. 2023, pp. 97–115.
- [50] N. E. H. Boubaker, K. Zarour, N. Guermouche, and D. Benmerzoug, "Optimizing workflow offloading and migration under timed constraints in fog and cloud computing," *J. Grid Comput.*, vol. 23, no. 1, p. 10, Mar. 2025.
- [51] C. Canali and R. Lancellotti, "A fog computing service placement for smart cities based on genetic algorithms," in *Proc. 9th Int. Conf. Cloud Comput. Services Sci.*, 2019, pp. 81–89.
- [52] M. K. Hussein and M. H. Mousa, "Efficient task offloading for IoT-based applications in fog computing using ant colony optimization," *IEEE Access*, vol. 8, pp. 37191–37201, 2020.
- [53] P. Ntumba, N. Georgantas, and V. Christophides, "Scheduling continuous operators for IoT edge analytics with time constraints," in *Proc. IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, Jun. 2022, pp. 78–85.
- [54] M. Pinedo and K. Hadavi, *Scheduling: Theory, Algorithms and Systems Development* (Operations Research Proceedings 1991). Cham, Switzerland: Springer, 1992, pp. 35–42.
- [55] R. N. Calheiros and R. Buyya, "Cost-effective provisioning and scheduling of deadline-constrained applications in hybrid clouds," in *Proc. WISE*, Jan. 2012, pp. 171–184.
- [56] F. Saeik, M. Avgeris, D. Spatharakis, N. Santi, D. Dechouniotis, J. Violos, A. Leivadeas, N. Athanasopoulos, N. Mitton, and S. Papavassiliou, "Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions," *Comput. Netw.*, vol. 195, Aug. 2021, Art. no. 108177.
- [57] C. Puliafito, E. Mingozzi, and G. Anastasi, "Fog computing for the Internet of Mobile Things: Issues and challenges," in *Proc. IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, May 2017, pp. 1–6.
- [58] K. Kumar, J. Liu, Y.-H. Lu, and B. Bhargava, "A survey of computation offloading for mobile systems," *Mobile Netw. Appl.*, vol. 18, no. 1, pp. 129–140, Feb. 2013.
- [59] S. M. Seyyedsalehi and M. Khansari, "Virtual machine placement optimization for big data applications in cloud computing," *IEEE Access*, vol. 10, pp. 96112–96127, 2022.
- [60] M. Ghobaei-Arani, A. Souiri, F. Safara, and M. Norouzi, "An efficient task scheduling approach using moth-flame optimization algorithm for cyber-physical system applications in fog computing," *Trans. Emerg. Telecommun. Technol.*, vol. 31, no. 2, p. 3770, Feb. 2020.
- [61] A. Lakhan, M. A. Mohammed, O. I. Obaid, C. Chakraborty, K. H. Abdulkareem, and S. Kadry, "Efficient deep-reinforcement learning aware resource allocation in SDN-enabled fog paradigm," *Autom. Softw. Eng.*, vol. 29, no. 1, pp. 1–25, May 2022.
- [62] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [63] K. Dolui and S. K. Datta, "Comparison of edge computing implementations: Fog computing, cloudlet and mobile edge computing," in *Proc. Global Internet Things Summit (GloTS)*, Jun. 2017, pp. 1–6.
- [64] Y. Ai, M. Peng, and K. Zhang, "Edge computing technologies for Internet of Things: A primer," *Digit. Commun. Netw.*, vol. 4, no. 2, pp. 77–86, Apr. 2018.
- [65] P. Hu, S. Dhelim, H. Ning, and T. Qiu, "Survey on fog computing: Architecture, key technologies, applications and open issues," *J. Netw. Comput. Appl.*, vol. 98, pp. 27–42, Nov. 2017.
- [66] Q. Qi, D. Zhao, T. W. Liao, and F. Tao, "Modeling of cyber-physical systems and digital twin based on edge computing, fog computing and cloud computing towards smart manufacturing," in *Proc. 13th Int. Manuf. Sci. Eng. Conf.*, vol. 1, 2018, pp. 1–7.
- [67] P. Mell and T. Grance, "The nist definition of cloud computing," Nat. Inst. Standards Technol. U.S. Dept. Commerce, Gaithersburg, MD, USA, Tech. Rep. NIST Special Publication 800-145, p. 7, 2011.
- [68] A. Mseddi, W. Jaafar, H. Elbiaze, and W. Ajib, "Joint container placement and task provisioning in dynamic fog computing," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10028–10040, Dec. 2019.
- [69] A. Kishor and C. Chakraborty, "Task offloading in fog computing for using smart ant colony optimization," *Wireless Pers. Commun.*, vol. 127, no. 2, pp. 1683–1704, Nov. 2022.
- [70] F. Hoseiny, S. Azizi, M. Shojafar, and R. Tafazolli, "Joint QoS-aware and cost-efficient task scheduling for fog-cloud resources in a volunteer computing system," 2021, *arXiv:2104.13974*.
- [71] S. Azizi, M. Shojafar, J. Abawajy, and R. Buyya, "Deadline-aware and energy-efficient IoT task scheduling in fog computing systems: A semi-greedy approach," *J. Netw. Comput. Appl.*, vol. 201, May 2022, Art. no. 103333.
- [72] T. S. Nikoui, A. Balador, A. M. Rahmani, and Z. Bakhshi, "Cost-aware task scheduling in fog-cloud environment," in *Proc. CSI/CPSSI Int. Symp. Real-Time Embedded Syst. Technol. (RTEST)*, Jun. 2020, pp. 1–8.
- [73] M. Yang, H. Ma, S. Wei, Y. Zeng, Y. Chen, and Y. Hu, "A multi-objective task scheduling method for fog computing in cyber-physical-social services," *IEEE Access*, vol. 8, pp. 65085–65095, 2020.
- [74] M. Salimian, M. Ghobaei-Arani, and A. Shahidinejad, "An evolutionary multi-objective optimization technique to deploy the IoT services in fog-enabled networks: An autonomous approach," *Appl. Artif. Intell.*, vol. 36, no. 1, pp. 1–34, Dec. 2022.
- [75] S. Nabi, M. Ahmad, M. Ibrahim, and H. Hamam, "AdPSO: Adaptive PSO-based task scheduling approach for cloud computing," *Sensors*, vol. 22, no. 3, p. 920, Jan. 2022.
- [76] M. Mtshali, S. Dlamini, M. Adigun, and P. Mudali, "K-means based on resource clustering for smart farming problem in fog computing," in *Proc. IEEE AFRICON*, Sep. 2019, pp. 1–6.
- [77] M. Abedi and M. Pourkiani, "Resource allocation in combined fog-cloud scenarios by using artificial intelligence," in *Proc. 5th Int. Conf. Fog Mobile Edge Comput. (FMEC)*, Apr. 2020, pp. 218–222.
- [78] R. K. Naha, S. Garg, S. K. Battula, M. B. Amin, and D. Georgakopoulos, "Multiple linear regression-based energy-aware resource allocation in the fog computing environment," 2021, *arXiv:2103.06385*.
- [79] A. Talpur and M. Gurusamy, "Reinforcement learning-based dynamic service placement in vehicular networks," in *Proc. IEEE 93rd Veh. Technol. Conf. (VTC-Spring)*, Apr. 2021, pp. 1–7.
- [80] C. Zhang and Z. Zheng, "Task migration for mobile edge computing using deep reinforcement learning," *Future Gener. Comput. Syst.*, vol. 96, pp. 111–118, Jul. 2019.
- [81] D. Lan, A. Taherkordi, F. Eliassen, Z. Chen, and L. Liu, "Deep reinforcement learning for intelligent migration of fog services in smart cities," in *Proc. Int. Conf. Algorithms Architectures Parallel Process.*, Jan. 2020, pp. 230–244.
- [82] J. Wang, J. Hu, G. Min, Q. Ni, and T. El-Ghazawi, "Online service migration in edge computing with incomplete information: A deep recurrent actor-critic method," 2020, *arXiv:2012.08679*.
- [83] Y. Chen, Z. Liu, Y. Zhang, Y. Wu, X. Chen, and L. Zhao, "Deep reinforcement learning-based dynamic resource management for mobile edge computing in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4925–4934, Jul. 2021.
- [84] J. Li, Z. Yang, X. Wang, Y. Xia, and S. Ni, "Task offloading mechanism based on federated reinforcement learning in mobile edge computing," *Digit. Commun. Netw.*, vol. 9, no. 2, pp. 492–504, Apr. 2023.

- [85] Y.-W. Zhang, W.-M. Zhang, K. Peng, D.-C. Yan, and Q.-L. Wu, "A novel edge server selection method based on combined genetic algorithm and simulated annealing algorithm," *Automatika*, vol. 62, no. 1, pp. 32–43, Jan. 2021.
- [86] N. Wang and B. Varghese, "Context-aware distribution of fog applications using deep reinforcement learning," 2020, *arXiv:2001.09228*.
- [87] J.-F. Tsai, C.-H. Huang, and M.-H. Lin, "An optimal task assignment strategy in cloud-fog computing environment," *Appl. Sci.*, vol. 11, no. 4, p. 1909, Feb. 2021.
- [88] G. Liu, J. Wang, Y. Tian, Z. Yang, and Z. Wu, "Mobility-aware dynamic service placement for edge computing," *EAI Endorsed Trans. Internet Things*, vol. 5, no. 19, p. e2, Jul. 2019.
- [89] F. R. Shahidani, A. Ghasemi, A. T. Haghighat, and A. Keshavarzi, "Task scheduling in edge-fog-cloud architecture: A multi-objective load balancing approach using reinforcement learning algorithm," *Computing*, vol. 105, no. 6, pp. 1–23, Jun. 2023.
- [90] Z. Sharif, L. Tang Jung, M. Ayaz, M. Yahya, and S. Pitafi, "Priority-based task scheduling and resource allocation in edge computing for health monitoring system," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 2, pp. 544–559, Feb. 2023.
- [91] S. Yang, G. Lee, and L. Huang, "Deep learning-based dynamic computation task offloading for mobile edge computing networks," *Sensors*, vol. 22, no. 11, p. 4088, May 2022.
- [92] X. Jie, T. Liu, H. Gao, C. Cao, P. Wang, and W. Tong, "A DQN-based approach for online service placement in mobile edge computing," in *Proc. Int. Conf. Collaborative Comput., Netw. Appl. Worksharing*, Jan. 2021, pp. 169–183.
- [93] M. K. Farimani, S. Karimian-Aliabadi, R. Entezari-Maleki, B. Egger, and L. Sousa, "Deadline-aware task offloading in vehicular networks using deep reinforcement learning," *Expert Syst. Appl.*, vol. 249, Sep. 2024, Art. no. 123622.
- [94] J. Liu, Y. Wang, D. Pan, and D. Yuan, "QoS-aware task offloading and resource allocation optimization in vehicular edge computing networks via MADDPG," *Comput. Netw.*, vol. 242, Apr. 2024, Art. no. 110282.
- [95] F. Liu, H. Yu, J. Huang, and T. Taleb, "Joint service migration and resource allocation in edge IoT system based on deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11341–11352, Apr. 2024.
- [96] S. Azizi, M. Shojafar, P. Farzin, and J. Dogani, "DCSP: A delay and cost-aware service placement and load distribution algorithm for IoT-based fog networks," *Comput. Commun.*, vol. 215, pp. 9–20, Feb. 2024.
- [97] S. Seifhosseini, M. Hosseini Shirvani, and Y. Ramzanpoor, "Multi-objective cost-aware bag-of-tasks scheduling optimization model for IoT applications running on heterogeneous fog environment," *Comput. Netw.*, vol. 240, Feb. 2024, Art. no. 110161.
- [98] O. S. Egwuche, J. Greeff, and A. E.-S. Ezugwu, "Optimized task offloading in multi-domain IoT networks using distributed deep reinforcement learning in edge computing environments," *IEEE Access*, vol. 13, pp. 26193–26207, 2025.
- [99] M. K. Jha and M. Kumar, "An autonomic offloading and resource allocation technique for IoT applications in edge computing," *J. Supercomput.*, vol. 81, no. 2, pp. 1–28, Jan. 2025.
- [100] N. S. Subramanian, P. Krishnan, K. Jain, K. B. A. Kumar, T. Pandey, and R. Buyya, "Blockchain and RL-based secured task offloading framework for software-defined 5G edge networks," *IEEE Access*, vol. 13, pp. 56820–56842, 2025.
- [101] A. R. Benamer, H. Teyeb, and N. B. Hadj-Alouane, "Latency-aware placement heuristic in fog computing environment," in *Proc. OTM Confederated Int. Conf. Move Meaningful Internet Syst.* Springer, 2018, pp. 241–257.
- [102] J. Sahoo, "Cost-efficient, QoS and security aware placement of smart farming IoT applications in cloud-fog infrastructure," 2021, *arXiv:2106.13524*.
- [103] M. Taneja and A. Davy, "Resource aware placement of IoT application modules in fog-cloud computing paradigm," in *Proc. IFIP/IEEE Symp. Integr. Netw. Service Manage. (IM)*, May 2017, pp. 1222–1228.
- [104] A. Ahmed Nacer, C. Godart, G. Rosinsky, A. Tari, and S. Youcef, "Business process outsourcing to the cloud: Balancing costs with security risks," *Comput. Ind.*, vol. 104, pp. 59–74, Jan. 2019.
- [105] Z. Rezazadeh, M. Rezaei, and M. Nickray, "LAMP: A hybrid fog-cloud latency-aware module placement algorithm for IoT applications," in *Proc. 5th Conf. Knowl. Based Eng. Innov. (KBEI)*, Feb. 2019, pp. 845–850.
- [106] S. Azizi, F. Khosroabadi, and M. Shojafar, "A priority-based service placement policy for fog-cloud computing systems," *Comput. Methods Differ. Equ.*, vol. 7, no. 4, pp. 521–534, Aug. 2019.
- [107] N. Mehran, D. Kimovski, and R. Prodan, "MAPO: A multi-objective model for IoT application placement in a fog environment," in *Proc. 9th Int. Conf. Internet Things*, Oct. 2019, pp. 1–8.
- [108] M. Sriraghavendra, P. Chawla, H. Wu, S. S. Gill, and R. Buyya, "Dosp: A deadline-aware dynamic service placement algorithm for workflow-oriented IoT applications in fog-cloud computing environments," in *Energy Conservation Solutions for Fog-Edge Computing Paradigms*. Springer, 2022, pp. 21–47.
- [109] X. Chen, S. Tang, Z. Lu, J. Wu, Y. Duan, S.-C. Huang, and Q. Tang, "IDISC: A new approach to IoT-data-intensive service components deployment in edge-cloud-hybrid system," *IEEE Access*, vol. 7, pp. 59172–59184, 2019.
- [110] L. Chen, Y. Xu, Z. Lu, J. Wu, K. Gai, P. C. K. Hung, and M. Qiu, "IoT microservice deployment in edge-cloud hybrid environment using reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12610–12622, Aug. 2021.
- [111] J. Paul Martin, A. Kandasamy, and K. Chandrasekaran, "CREW: Cost and reliability aware eagle-whale optimiser for service placement in fog," *Softw., Pract. Exper.*, vol. 50, no. 12, pp. 2337–2360, Dec. 2020.
- [112] A. M. Yadav and S. C. Sharma, "Cooperative task scheduling secured with blockchain in sustainable mobile edge computing," *Sustain. Comput., Informat. Syst.*, vol. 37, Jan. 2023, Art. no. 100843.
- [113] M. I. Khaleel, M. Safran, S. Alfahhood, and D. Gupta, "Combinatorial Metaheuristic methods to optimize the scheduling of scientific workflows in green DVFS-enabled edge-cloud computing," *Alexandria Eng. J.*, vol. 86, pp. 458–470, Jan. 2024.
- [114] M. Shiva Rama Krishna and D. Khasim Vali, "ADWEH: A dynamic prioritized workflow task scheduling approach based on the enhanced Harris hawk optimization algorithm," *IEEE Access*, vol. 13, pp. 35490–35515, 2025.
- [115] P. Choppa and S. S. Mangalampalli, "Resource adaptive automated task scheduling using deep deterministic policy gradient in fog computing," *IEEE Access*, vol. 13, pp. 25969–25994, 2025.
- [116] L. Jourdan, M. Basseur, and E.-G. Talbi, "Hybridizing exact methods and metaheuristics: A taxonomy," *Eur. J. Oper. Res.*, vol. 199, no. 3, pp. 620–629, Dec. 2009.
- [117] K. Rajwar, K. Deep, and S. Das, "An exhaustive review of the Metaheuristic algorithms for search and optimization: Taxonomy, applications, and open challenges," *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 13187–13257, Nov. 2023.
- [118] I. Boussaïd, J. Lepagnot, and P. Siarry, "A survey on optimization metaheuristics," *Inf. Sci.*, vol. 237, pp. 82–117, Jul. 2013.
- [119] M. Morales, *Grokking Deep Reinforcement Learning*. Shelter Island, NY, USA: Manning Publications, 2020.
- [120] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "IFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments," *Softw., Pract. Exper.*, vol. 47, no. 9, pp. 1275–1296, Sep. 2017.
- [121] A. Seyyedabbasi, R. Aliyev, F. Kiani, M. U. Gulle, H. Basyildiz, and M. A. Shah, "Hybrid algorithms based on combining reinforcement learning and Metaheuristic methods to solve global optimization problems," *Knowl.-Based Syst.*, vol. 223, Jul. 2021, Art. no. 107044.
- [122] E.-G. Talbi, "Combining metaheuristics with mathematical programming, constraint programming and machine learning," *Ann. Oper. Res.*, vol. 240, no. 1, pp. 171–215, 2016.
- [123] L. Calvet, J. D. Armas, D. Masip, and A. A. Juan, "Learnheuristics: Hybridizing metaheuristics with machine learning for optimization with dynamic inputs," *Open Math.*, vol. 15, no. 1, pp. 261–280, Mar. 2017.
- [124] W. Zhu, X. Liu, M. Xu, and H. Wu, "Predicting the results of RNA molecular specific hybridization using machine learning," *IEEE/CAA J. Autom. Sinica*, vol. 6, no. 6, pp. 1384–1396, Nov. 2019.
- [125] A. Aldoseri, K. N. Al-Khalifa, and A. M. Hamouda, "Re-thinking data strategy and integration for artificial intelligence: Concepts, opportunities, and challenges," *Appl. Sci.*, vol. 13, no. 12, p. 7082, Jun. 2023.
- [126] A. Mseddi, W. Jaafar, H. Elbiaze, and W. Ajib, "Intelligent resource allocation in dynamic fog computing environments," in *Proc. IEEE 8th Int. Conf. Cloud Netw. (CloudNet)*, Nov. 2019, pp. 1–7.
- [127] S. Zhou, W. Jadoon, and J. Shuja, "Machine learning-based offloading strategy for lightweight user mobile edge computing tasks," *Complexity*, vol. 2021, no. 1, pp. 1–11, Jan. 2021.

- [128] A. Ometov, O. Molua, M. Komarov, and J. Nurmi, "A survey of security in cloud, edge, and fog computing," *Sensors*, vol. 22, no. 3, p. 927, Jan. 2022.
- [129] H. Xu, P. V. Klaine, O. Onireti, B. Cao, M. Imran, and L. Zhang, "Blockchain-enabled resource management and sharing for 6G communications," *Digit. Commun. Netw.*, vol. 6, no. 3, pp. 261–269, Aug. 2020.
- [130] M. S. Mahmood and N. B. Al Dabagh, "Blockchain technology and Internet of Things: Review, challenge and security concern," *Int. J. Electr. Comput. Eng.*, vol. 13, no. 1, p. 718, Feb. 2023.
- [131] L. Vu, Q. U. Nguyen, D. N. Nguyen, D. T. Hoang, and E. Dutkiewicz, "Deep transfer learning for IoT attack detection," *IEEE Access*, vol. 8, pp. 107335–107344, 2020.
- [132] Z. Liu, L. Qian, and S. Tang, "The prediction of DDoS attack by machine learning," *Proc. SPIE*, vol. 12167, pp. 681–686, Mar. 2022.
- [133] L. Heng, G. Yin, and X. Zhao, "Energy aware cloud-edge service placement approaches in the Internet of Things Communications," *Int. J. Commun. Syst.*, vol. 35, no. 1, p. 4899, Jan. 2022.
- [134] C.-H. Hong and B. Varghese, "Resource management in fog/edge computing: A survey on architectures, infrastructure, and algorithms," *ACM Comput. Surv.*, vol. 52, no. 5, pp. 1–37, Sep. 2020.
- [135] M. Heck, J. Edinger, D. Schaefer, and C. Becker, "IoT applications in fog and edge computing: Where are we and where are we going?" in *Proc. 27th Int. Conf. Comput. Commun. Netw. (ICCCN)*, Jul. 2018, pp. 1–6.
- [136] Z. Á. Mann, "Notions of architecture in fog computing," *Computing*, vol. 103, no. 1, pp. 51–73, 2021.
- [137] J. Bellendorf and Z. Á. Mann, "Classification of optimization problems in fog computing," *Future Gener. Comput. Syst.*, vol. 107, pp. 158–176, Jun. 2020.
- [138] E. Badidi, "QoS-aware placement of tasks on a fog cluster in an edge computing environment," *J. Ubiquitous Syst. Pervasive Netw.*, vol. 13, no. 1, pp. 11–19, Oct. 2020.
- [139] A. Mijuskovic, A. Chiumento, R. Bemthuis, A. Aldea, and P. Havinga, "Resource management techniques for cloud/fog and edge computing: An evaluation framework and classification," *Sensors*, vol. 21, no. 5, p. 1832, Mar. 2021.
- [140] E. N. Alkhanak, S. P. Lee, and S. U. R. Khan, "Cost-aware challenges for workflow scheduling approaches in cloud computing environments: Taxonomy and opportunities," *Future Gener. Comput. Syst.*, vol. 50, pp. 3–21, Sep. 2015.



NOUR EL HOUDA BOUBAKER received the bachelor's and master's degrees in information systems and web technologies, in 2017 and 2019, respectively, and the Ph.D. degree in computer science from the Faculty of New Technologies of Information and Communication, University of Constantine 2–Abdelhamid Mehri, Algeria. Her research focuses on cyber-physical business process management, fog and cloud computing, and machine learning techniques. She has published work in these areas.



KARIM ZAROUR received the Ph.D. degree from the Faculty of New Technologies of Information and Communication, University of Constantine 2–Abdelhamid Mehri, Algeria, in 2020. Currently, he holds the position of an Associate Professor with the University of Constantine 2–Abdelhamid Mehri. His research interest primarily includes business process management in fog and cloud computing environments.



NAWAL GUERMOUCHE received the Ph.D. degree in computer science from LORIA-INRIA, University of Lorraine, Nancy, France, in 2010. She has been an Associate Professor with the National Institute of Applied Sciences (INSA), since 2011. She is a member of LAAS-CNRS, the National Center for Scientific Research. She has published several research papers in well-known international conferences and journals. She is involved in several national and international research and development projects for a wide range of applications, including service-oriented systems and the IoT. Her research interests include service-oriented architecture and computing, business process management, cloud/fog-based architectures, the Internet of Things, and cyberphysical systems. She is a member of program committees of several international conferences and journals.



DJAMEL BENMERZOUG received the Ph.D. degree in computer science from Pierre and Marie Curie University, Paris, France. He is currently a Full Professor with the Department of TLSI, Faculty of New Technologies of Information and Communication, University of Constantine 2–Abdelhamid Mehri, Algeria. He has published many articles in many international conferences and journals. He has supervised many Ph.D. and master's students. His current research interests include the Internet of Things, cloud computing, advanced enterprises systems, multiagent systems, service oriented computing, and business processes modeling and verification.

...